Kiến Trúc Giải Pháp — Web Crawler & Ingestion Pipeline trên AWS

> ***Phiên bản:*** *1.0* | ***Môi trường:*** `demo` | ***Prefix tài nguyên:*** `crawler-demo-*` ***Phạm vi:*** *Scope 1 — Production-ready, chi phí kiểm soát, quy mô nhỏ* ***Ngôn ngữ IaC:*** *Terraform ≥ 1.5* | ***Config Management:*** *Ansible* | ***Runtime:*** *Python 3.12*

Mục lục

1. Phạm vi bài toán
2. Kiến trúc tổng quan
3. Luồng dữ liệu end-to-end
4. Phân tích từng dịch vụ
5. Mạng & bảo mật
6. Infrastructure as Code — Terraform
7. Ansible — cấu hình runtime
8. CI/CD
9. Hoạch định năng lực & toán cấu hình
10. High Availability & Fault Tolerance
11. Scale-up scenarios
12. Lỗi thường gặp & troubleshooting
13. Observability chi tiết
14. Checklist vận hành
15. Phụ lục — Quick Reference Map

---

1. Phạm vi bài toán

1.1 Mục tiêu hệ thống

Hệ thống thu thập nội dung bài viết từ các nguồn web (RSS feed và Sitemap XML), chuẩn hóa dữ liệu, lưu trữ vào cơ sở dữ liệu quan hệ PostgreSQL, và cung cấp giao diện web để tra cứu. Toàn bộ pipeline chạy trên AWS với chi phí kiểm soát ở mức Scope 1 (Free Tier hoặc gần Free Tier).

1.2 Loại dữ liệu và nguồn crawl

Trường dữ liệu	Kiểu	Ràng buộc	Ghi chú
`id`	BIGSERIAL	PRIMARY KEY	Auto-increment
`source`	VARCHAR(120)	NOT NULL	Tên/domain nguồn
`canonical_url`	VARCHAR(2048)	NOT NULL, UNIQUE	URL chuẩn hóa — khóa idempotency
`title`	VARCHAR(512)	nullable	Tiêu đề bài viết
`summary`	TEXT	nullable	Tóm tắt / mô tả
`published_at`	TIMESTAMPTZ	nullable	Thời điểm xuất bản gốc

`fetched_at`	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Thời điểm hệ thống thu thập

Nguồn crawl được hỗ trợ:

- RSS 2.0 / Atom feeds
- Sitemap XML (sitemap index + sitemap URL)
- Giới hạn: `max_items_per_source = 100` bài/nguồn/chu kỳ

1.3 Cơ chế thu thập

Worker chạy theo lịch `APScheduler` với chu kỳ `crawler_interval_seconds = 1800` (30 phút). Mỗi chu kỳ:

1. Đọc danh sách nguồn từ cấu hình
2. Fetch RSS/Sitemap, parse, chuẩn hóa thành danh sách article dict
3. Nếu payload ≤ 200 KiB → gửi inline JSON array vào SQS
4. Nếu payload > 200 KiB → upload lên S3 raw bucket, gửi Claim Check pointer vào SQS

1.4 Giới hạn kỹ thuật (Scope 1)

Tham số	Giá trị	Lý do
Worker instances	1–2 (ASG)	Chi phí t3.micro
Lambda concurrency	tối đa 5 (ESM)	Bảo vệ RDS t3.micro
RDS	db.t3.micro, Single-AZ	Free Tier / chi phí
Crawl interval	30 phút	Tránh bị block bởi nguồn
Items/source/cycle	100	Giới hạn throughput
S3 raw retention	30 ngày	Tiết kiệm storage
ECR images giữ lại	10	Tiết kiệm storage

1.5 Các kịch bản vận hành (A–E)

Kịch bản	Mô tả	Xử lý
A — Normal	Payload ≤ 200 KiB, Lambda xử lý thành công	Inline JSON → SQS → Lambda → INSERT → commit
B — Large payload	Payload > 200 KiB (nhiều bài/nguồn lớn)	Claim Check: S3 upload → SQS pointer → Lambda fetch S3 → INSERT
C — Duplicate URL	URL đã tồn tại trong DB	`ON CONFLICT DO NOTHING` — bỏ qua, không lỗi, không duplicate
D — Lambda failure	Exception trong Lambda (DB down, parse error)	`ReportBatchItemFailures` → chỉ record lỗi quay lại SQS, tối đa 3 lần → DLQ
E — Worker crash	EC2 instance bị terminate	ASG tự thay thế instance mới trong AZ khác; systemd `Restart=always`

1.6 Out-of-scope (Scope 1)

- Full-text search (Elasticsearch/OpenSearch)
- Multi-region deployment
- HTTPS/TLS trên ALB (chỉ HTTP port 80)
- Authentication/Authorization cho dashboard
- Real-time streaming (Kinesis)
- Deduplication ở tầng SQS (dùng FIFO)
- Automated schema migration (Flyway/Alembic)

---

2. Kiến trúc tổng quan

2.1 Sơ đồ kiến trúc ASCII

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  AWS Region: ap-southeast-1                                                  │
│                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │  VPC: 10.0.0.0/16  (crawler-demo-vpc)                                │    │
│  │                                                                      │    │
│  │  ┌──────────────────────┐     ┌──────────────────────┐              │    │
│  │  │  Public Subnet AZ-a  │     │  Public Subnet AZ-b  │              │    │
│  │  │  10.0.1.0/24         │     │  10.0.2.0/24         │              │    │
│  │  │  ┌────────────────┐   │     │                      │              │    │
│  │  │  │  NAT Gateway   │   │     │  ┌────────────────┐  │              │    │
│  │  │  │  (Elastic IP)  │   │     │  │  ALB (web)     │  │              │    │
│  │  │  └────────┬───────┘   │     │  └────────┬───────┘  │              │    │
│  │  └───────────┼───────────┘     └───────────┼──────────┘              │    │
│  │              │                             │                         │    │
│  │              │ egress                      │ HTTP:80                 │    │
│  │  ┌───────────┼─────────────────────────────┼──────────┐              │    │
│  │  │  Private Subnet AZ-a  10.0.11.0/24      │          │              │    │
│  │  │  ┌──────────────────────────────────┐   │          │              │    │
│  │  │  │  EC2 Worker (t3.micro)           │◄──┘          │              │    │
│  │  │  │  Docker: crawler-worker          │              │              │    │
│  │  │  │  Docker: crawler-web :8080       │              │              │    │
│  │  │  │  systemd + CW Agent              │              │              │    │
│  │  │  └──────────────┬───────────────────┘              │              │    │
│  │  └─────────────────┼──────────────────────────────────┘              │    │
│  │                    │                                                 │    │
│  │  ┌─────────────────┼──────────────────────────────────┐              │    │
│  │  │  Private Subnet AZ-b  10.0.12.0/24                 │              │    │
│  │  │  ┌──────────────────────────────────┐              │              │    │
│  │  │  │  Lambda ENI (VPC-attached)       │              │              │    │
│  │  │  │  crawler-demo-ingester           │              │              │    │
│  │  │  └──────────────┬───────────────────┘              │              │    │
│  │  └─────────────────┼──────────────────────────────────┘              │    │
│  │                    │                                                 │    │
│  │  ┌─────────────────┼──────────────────────────────────────────┐      │    │
│  │  │  DB Subnet  10.0.21.0/24 + 10.0.22.0/24                    │      │    │
│  │  │  ┌──────────────────────────────────────────────────┐      │      │    │
│  │  │  │  RDS PostgreSQL 15 (db.t3.micro, Single-AZ)      │      │      │    │
```

```
│  │  │  │  crawler-demo-db  port 5432                          │   │   │   │
│  │  │  └──────────────────────────────────────────────────────┘   │   │   │
│  │  └──────────────────────────────────────────────────────────  │   │   │
│  └──────────────────────────────────────────────────────────────┘   │   │
│                                                                      │   │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  AWS Managed Services (Regional)                              │   │
│  │                                                               │   │
│  │  SQS Standard Queue          S3 Gateway VPC Endpoint          │   │
│  │  crawler-demo-data-queue  ──►  com.amazonaws.*.s3             │   │
│  │  crawler-demo-data-dlq                                        │   │
│  │                                                               │   │
│  │  S3 Buckets                  KMS CMK                          │   │
│  │  crawler-demo-raw-*          alias/crawler-demo-key           │   │
│  │  crawler-demo-exports-*      (rotation enabled)               │   │
│  │                                                               │   │
│  │  Secrets Manager             ECR Repository                   │   │
│  │  crawler-demo/db-credentials crawler-demo-worker              │   │
│  │                                                               │   │
│  │  CloudWatch Logs + Alarms + Dashboard                         │   │
│  │  SNS Topic: crawler-demo-alerts                               │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘


Luồng dữ liệu chính:
  Internet → Worker (NAT) → SQS → Lambda → RDS PostgreSQL
                                         → S3 raw (Claim Check)
  Internet → ALB → Worker (crawler-web) → RDS (read-only dashboard)
```

2.2 Nguyên tắc thiết kế

Loose Coupling (Tách rời): Worker và Lambda không giao tiếp trực tiếp. SQS đóng vai trò buffer bất đồng bộ — Worker có thể crash mà không mất message; Lambda có thể chậm mà không block Worker.

Idempotency (Bất biến khi lặp lại): `INSERT ... ON CONFLICT (canonical_url) DO NOTHING` đảm bảo gửi lại message không tạo duplicate. Lambda có thể được invoke nhiều lần cho cùng một message mà không gây hại.

Governor Pattern (Kiểm soát tốc độ): `maximum_concurrency = 5` trên ESM giới hạn số Lambda instance đồng thời, bảo vệ RDS t3.micro khỏi connection exhaustion. Đây là "governor" — không phải throttle cứng mà là soft cap có thể điều chỉnh.

Defense in Depth (Bảo vệ nhiều lớp): KMS mã hóa at-rest (RDS, SQS, S3, Secrets Manager), TLS-only SQS policy, IMDSv2 bắt buộc, Security Groups theo nguyên tắc least-privilege, IAM roles tách biệt cho Worker và Lambda.

Claim Check Pattern: Tránh giới hạn 256 KiB của SQS bằng cách offload payload lớn lên S3, chỉ gửi pointer qua SQS. Ngưỡng: 200 KiB (`claim_check_threshold_bytes = 204800`).

2.3 Danh mục thành phần → Module Terraform

Thành phần	Module Terraform	Tài nguyên chính

VPC, Subnets, NAT, S3 Endpoint	`modules/networking`	`aws_vpc`, `aws_subnet`, `aws_nat_gateway`, `aws_vpc_endpoint`
KMS, Secrets Manager, SG, IAM	`modules/security`	`aws_kms_key`, `aws_secretsmanager_secret`, `aws_security_group`, `aws_iam_role`
SQS Main + DLQ	`modules/queue`	`aws_sqs_queue` × 2, `aws_sqs_queue_policy` × 2
RDS PostgreSQL, S3 raw, S3 exports	`modules/storage`	`aws_db_instance`, `aws_s3_bucket` × 2
Lambda Function, Layer, ESM	`modules/lambda`	`aws_lambda_function`, `aws_lambda_layer_version`, `aws_lambda_event_source_mapping`
ECR, Launch Template, ASG	`modules/worker`	`aws_ecr_repository`, `aws_launch_template`, `aws_autoscaling_group`
SNS, CW Alarms, Dashboard	`modules/observability`	`aws_sns_topic`, `aws_cloudwatch_metric_alarm` × 4, `aws_cloudwatch_dashboard`
ALB, Target Group, Listener	`environments/demo/main.tf`	`aws_lb`, `aws_lb_target_group`, `aws_lb_listener`

---

3. Luồng dữ liệu end-to-end

3.1 Sơ đồ luồng chi tiết

```
[Nguồn RSS/Sitemap]
        │
        │ HTTP GET (qua NAT Gateway)
        ▼
┌──────────────────────────────────────────────────────┐
│  EC2 Worker (crawler-demo-worker-asg)                 │
│  Container: crawler-worker (Docker)                   │
│                                                       │
│  1. APScheduler trigger (mỗi 1800s)                   │
│  2. Fetch RSS/Sitemap → parse → normalize             │
│  3. Serialize thành JSON list                         │
│  4. len(payload) > 204800 bytes?                      │
│     YES → PutObject S3 raw bucket                     │
│            body = {"claim_check_s3_bucket":...,       │
│                    "claim_check_s3_key":...}          │
│     NO  → body = [{article1}, {article2}, ...]        │
│  5. SQS SendMessage (TLS, KMS encrypted)              │
└──────────────────────────────────────────────────────┘
        │
        │ SQS Standard Queue
        │ crawler-demo-data-queue
        │ VT=1080s, maxReceiveCount=3
        ▼
┌──────────────────────────────────────────────────────┐
│  Lambda Ingester (crawler-demo-ingester)              │
```

```
│  Trigger: SQS ESM, BatchSize=10, max_concurrency=5    │
│                                                       │
│  Per invocation (1 batch = ≤10 records):              │
│  6.  _get_connection() → reuse warm conn hoặc         │
│      pg8000.connect() + _ensure_schema() (cold start) │
│  7.  For each SQS record:                             │
│      a. _resolve_payload(body):                       │
│         - inline list → dùng trực tiếp               │
│         - claim_check → S3 GetObject → decompress     │
│      b. For each article in list:                     │
│         INSERT INTO articles ... ON CONFLICT DO NOTHING│
│         cursor.rowcount==1 → inserted; ==0 → skipped  │
│      c. conn.commit()                                 │
│      d. log record_processed (JSON structured)        │
│      e. Exception → conn.rollback()                   │
│                   → batch_item_failures.append(id)    │
│  8.  log batch_summary                                │
│  9.  return {"batchItemFailures": [...]}              │
└───────────────────────────────────────────────────────┘
        │                    │
        │ Thành công          │ Thất bại (≤3 lần)
        ▼                    ▼
┌──────────────┐     ┌──────────────────────┐
│  RDS         │     │  SQS DLQ             │
│  PostgreSQL  │     │  crawler-demo-       │
│  articles    │     │  data-dlq            │
│  table       │     │  → SNS Alert Email   │
└──────────────┘     └──────────────────────┘
```

3.2 Bước-by-bước chi tiết

Bước 1 — Scheduler trigger: APScheduler trong container `crawler-worker` kích hoạt mỗi `CRAWLER_INTERVAL_SECONDS=1800`. Biến này được inject qua environment variable trong systemd unit (Ansible template `crawler-worker.service.j2` hoặc `user_data.sh.tpl`).

Bước 2 — Fetch & Parse: Worker fetch URL nguồn qua HTTP. Traffic đi qua NAT Gateway (single NAT, Scope 1). Parse RSS/Atom/Sitemap XML, extract fields: `source`, `canonical_url`, `title`, `summary`, `published_at`. Giới hạn `max_items_per_source=100`.

Bước 3 — Serialize: Chuẩn hóa thành Python list of dicts. Serialize thành JSON bytes.

Bước 4 — Claim Check decision:

```
if len(payload_bytes) > claim_check_threshold_bytes:  # 204800
    s3.put_object(Bucket=raw_bucket, Key=f"{source}/{uuid}.json.gz",
Body=gzip(payload))
    message_body = json.dumps({"claim_check_s3_bucket": bucket,
"claim_check_s3_key": key})
else:
    message_body = json.dumps(articles_list)
```

S3 traffic đi qua **S3 Gateway VPC Endpoint** — không qua NAT, không tốn phí NAT data processing.

Bước 5 — SQS SendMessage: Worker gọi `sqs:SendMessage` với IAM role `crawler-demo-worker-role`. Message được mã hóa bằng KMS CMK (`alias/crawler-demo-key`). Queue policy từ chối mọi request không dùng TLS (`aws:SecureTransport = false`).

Bước 6 — Lambda cold/warm start:

- **Cold start:** Lambda container khởi tạo, `_get_connection()` tạo kết nối pg8000 mới, `_ensure_schema()` chạy DDL idempotent (`CREATE TABLE IF NOT EXISTS` , `CREATE UNIQUE INDEX IF NOT EXISTS`). Kết nối được lưu vào global `_conn` .
- **Warm start:** `_conn` đã tồn tại, thực hiện `SELECT 1` để kiểm tra liveness. Nếu connection bị đóng (RDS idle timeout), reconnect tự động.

Bước 7 — Claim Check resolution:

```
def _resolve_payload(body: str) -> list[dict]:
    doc = json.loads(body)
    if isinstance(doc, list):
        return doc  # inline
    if "claim_check_s3_key" in doc:
        obj = s3.get_object(Bucket=doc["claim_check_s3_bucket"],
Key=doc["claim_check_s3_key"])
        data = obj["Body"].read()
        if obj.get("ContentEncoding") == "gzip" or key.endswith(".gz"):
            data = gzip.decompress(data)
        return json.loads(data)
```

Bước 8 — INSERT với idempotency:

```
INSERT INTO articles (source, canonical_url, title, summary, published_at,
fetched_at)
VALUES (%s, %s, %s, %s, %s, %s)
ON CONFLICT (canonical_url) DO NOTHING
```

`cursor.rowcount == 1` → inserted; `== 0` → duplicate, bỏ qua. Không có SELECT probe trước — atomic, không TOCTOU race.

Bước 9 — Partial batch failure: Lambda trả về `{"batchItemFailures": [{"itemIdentifier": message_id}, ...]}` . SQS chỉ xóa các record thành công; record thất bại quay lại queue với `ReceiveCount` tăng. Sau 3 lần thất bại → DLQ.

3.3 Map code → Terraform resource

Hành động	Code/Config	Terraform Resource
Worker gửi SQS	`CRAWLER_SQS_QUEUE_URL` env var	`module.queue.main_queue_url` → `module.worker`
Worker upload S3	`CRAWLER_S3_RAW_BUCKET` env var	`module.storage.s3_raw_bucket` → `module.worker`
Lambda trigger	ESM `aws_lambda_event_source_mapping.sqs`	`modules/lambda/main.tf`

Lambda đọc S3	IAM `S3RawReadClaimCheck`	`modules/security/main.tf` → `aws_iam_role_policy.lambda_custom`
Lambda kết nối RDS	`RDS_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` env vars	`module.storage.rds_endpoint` → `module.lambda`
Lambda VPC access	`vpc_config` trong `aws_lambda_function`	`modules/lambda/main.tf`
DLQ routing	`redrive_policy.maxReceiveCount=3`	`modules/queue/main.tf` → `aws_sqs_queue.main`

---

4. Phân tích từng dịch vụ

4.1 EC2 ASG Worker

Lý do chọn (so sánh alternatives)

Tiêu chí	EC2 ASG ✅	ECS Fargate	Lambda Scheduled
Chi phí	t3.micro Free Tier	~$0.04/vCPU-hr, không Free Tier	Rất rẻ nếu chạy ngắn
Trạng thái	Stateful (có thể cache)	Stateless	Stateless
Crawl duration	Không giới hạn	Không giới hạn	Max 15 phút
Docker support	Có (Docker CE)	Native	Không (container image có)
Complexity	Trung bình (Ansible)	Cao (ECS task def, service)	Thấp
Scope 1 fit	✅ Tốt nhất	Overkill	Giới hạn timeout
SSH/SSM access	Có (debug dễ)	Exec command	Không

Quyết định: EC2 ASG phù hợp nhất cho Scope 1 vì: (1) t3.micro nằm trong Free Tier, (2) crawl job có thể chạy lâu hơn 15 phút nếu nhiều nguồn, (3) dễ debug qua SSM Session Manager, (4) Ansible quản lý config linh hoạt hơn ECS task definition.

Nguyên lý hoạt động

- **Amazon Linux 2023 (AL2023):** AMI được chọn tự động qua `data "aws_ami" "al2023"` với filter `al2023-ami-2023.*-kernel-6.1-x86_64`. Luôn dùng AMI mới nhất khi launch template được refresh.
- **IMDSv2 bắt buộc:** `http_tokens = "required"` trong `metadata_options` — ngăn SSRF attack lấy credentials từ IMDS.
- **Docker CE:** Cài qua Ansible role `docker` (hoặc `user_data.sh.tpl` cho bootstrap lần đầu). Container chạy dưới systemd với `Restart=always`.
- **Multi-AZ:** ASG span 2 private subnets (AZ-a: `10.0.11.0/24`, AZ-b: `10.0.12.0/24`). Nếu AZ-a down, ASG launch instance mới ở AZ-b.

Cơ chế cụ thể trong project

```
crawler-demo-worker-asg
  desired=1, min=1, max=2
  Launch Template: crawler-demo-worker-lt
    AMI: al2023-ami-2023.*
    Instance type: t3.micro
    IAM Profile: crawler-demo-worker-profile
    SG: crawler-demo-sg-worker (egress all + ingress :8080 từ ALB)
    IMDSv2: required
    EBS: 8GB gp3, encrypted
    User data: user_data.sh.tpl (bootstrap Docker + systemd)

Scaling policies:
  Scale-out: CPU > 70% trong 3 phút → +1 instance (cooldown 180s)
  Scale-in:  CPU < 40% trong 3 phút → -1 instance (cooldown 180s)
```

Hai container chạy trên mỗi instance:

1. `crawler-worker` : APScheduler crawl loop, gửi SQS
2. `crawler-web` : FastAPI dashboard, đọc RDS, expose port 8080

Log forwarding: `crawler-log-forward.service` (systemd) chạy script `crawler-forward-docker-logs.sh` — tail Docker logs → `/var/log/crawler.log` → CloudWatch Agent ship lên `/ec2/crawler-demo-worker` .

Map Terraform resource

Resource	File
`aws_ecr_repository.worker`	`modules/worker/main.tf`
`aws_launch_template.worker`	`modules/worker/main.tf`
`aws_autoscaling_group.worker`	`modules/worker/main.tf`
`aws_autoscaling_policy.scale_out/in`	`modules/worker/main.tf`
`aws_cloudwatch_metric_alarm.cpu_high/low`	`modules/worker/main.tf`
`aws_cloudwatch_log_group.worker`	`modules/worker/main.tf`
`aws_iam_role.worker` + profile	`modules/security/main.tf`

Health check & Logging

- **EC2 health check:** ASG dùng `health_check_type = "EC2"` — instance bị đánh dấu unhealthy nếu EC2 status check fail.
- **ALB health check:** `GET /health` → HTTP 200, interval 30s, threshold 2/2.
- **Log group:** `/ec2/crawler-demo-worker` , retention 30 ngày.
- **Log streams:** `{instance_id}/user-data` , `{instance_id}/crawler` .

4.2 SQS Standard Queue

Lý do chọn (so sánh alternatives)

Tiêu chí	SQS Standard ✅	SQS FIFO	Kinesis	EventBridge
Throughput	Gần vô hạn	300 msg/s/group	1MB/s/shard	Có giới hạn
Ordering	Best-effort	Strict	Per-shard	N/A
Dedup	Không (dùng DB)	Content-based	N/A	N/A
At-least-once	Có	Có	Có	Có
DLQ	Có	Có	Có	Có
Chi phí	Rẻ nhất	Rẻ	Đắt hơn	Đắt hơn
Lambda ESM	Có	Có	Có	Có

Quyết định: SQS Standard đủ dùng vì idempotency được xử lý ở tầng DB (`ON CONFLICT DO NOTHING`). Ordering không cần thiết — mỗi message là một batch độc lập. Throughput của Standard cao hơn FIFO nhiều lần, phù hợp khi scale.

Nguyên lý hoạt động

- **Visibility Timeout (VT):** Khi Lambda nhận message, SQS ẩn message đó trong `VT = 1080s` . Nếu Lambda xử lý xong và trả về trước VT → SQS xóa message. Nếu Lambda crash hoặc timeout → message tái xuất hiện sau VT.
- **At-least-once delivery:** SQS có thể deliver cùng message nhiều lần (đặc biệt khi VT quá ngắn). Idempotency ở DB là safety net.
- **DLQ:** Sau `maxReceiveCount = 3` lần nhận mà không xóa → message chuyển sang DLQ `crawler-demo-data-dlq` .
- **KMS encryption:** Message được mã hóa bằng CMK `alias/crawler-demo-key` trước khi lưu vào SQS storage.
- **TLS-only policy:** Queue policy deny mọi action nếu `aws:SecureTransport = false` .

Cơ chế cụ thể trong project

```
crawler-demo-data-queue (Standard)
  visibility_timeout_seconds = 1080  # 6 × Lambda timeout 180s
  message_retention_seconds  = 345600  # 4 ngày (default)
  receive_wait_time_seconds  = 20  # Long polling
  kms_master_key_id          = alias/crawler-demo-key
  redrive_policy:
    deadLetterTargetArn = crawler-demo-data-dlq.arn
    maxReceiveCount     = 3

crawler-demo-data-dlq (Standard)
  visibility_timeout_seconds = 1080
  message_retention_seconds  = 1209600  # 14 ngày
  kms_master_key_id          = alias/crawler-demo-key
```

Map Terraform resource

Resource	File

`aws_sqs_queue.main`	`modules/queue/main.tf`
`aws_sqs_queue.dlq`	`modules/queue/main.tf`
`aws_sqs_queue_policy.main`	`modules/queue/main.tf`
`aws_sqs_queue_policy.dlq`	`modules/queue/main.tf`

4.3 Lambda Ingester

Lý do chọn (so sánh alternatives)

Tiêu chí	Lambda ✅	ECS Task (consumer)	EC2 Consumer
Chi phí	Pay-per-invocation	Pay-per-task-hour	Pay-per-instance-hour
Scale	Auto (ESM)	Manual/ECS Service	Manual/ASG
Cold start	~200ms (VPC)	N/A	N/A
Timeout	15 phút max	Không giới hạn	Không giới hạn
SQS integration	Native ESM	Polling tự viết	Polling tự viết
Managed	Fully managed	Partially	Không
Scope 1 fit	✅ Tốt nhất	Overkill	Overkill

Quyết định: Lambda với SQS ESM là pattern chuẩn cho event-driven ingestion. Không cần quản lý infrastructure, tự scale theo queue depth, tích hợp `ReportBatchItemFailures` native.

Nguyên lý hoạt động

Runtime: Python 3.12, memory 256MB, timeout 180s.

Layer: `pg8000` pure-Python PostgreSQL driver được đóng gói thành Lambda Layer (`postgres_pure_layer.zip`). Không cần `psycopg2` với native binaries — đơn giản hóa packaging.

VPC attachment: Lambda chạy trong private subnets với `sg_lambda` (egress all). ENI được tạo trong private subnets để kết nối RDS trong db subnets.

ESM configuration:

```
batch_size                          = 10
maximum_batching_window_in_seconds = 30  # chờ tối đa 30s để gom đủ 10 records
function_response_types             = ["ReportBatchItemFailures"]
scaling_config.maximum_concurrency = 5  # governor: tối đa 5 Lambda đồng thời
```

Connection reuse (warm start):

```
_conn = None  # global, tồn tại suốt lifetime của container

def _get_connection():
    global _conn
    if _conn is not None:
```

```
        try:
            _conn.run("SELECT 1")  # liveness check
            return _conn
        except Exception:
            _conn = None  # reconnect
    _conn = pg8000.connect(...)
    _ensure_schema(_conn)
    return _conn
```

Schema bootstrap (cold start): `_ensure_schema()` chạy `CREATE TABLE IF NOT EXISTS` và `CREATE UNIQUE INDEX IF NOT EXISTS` — idempotent, an toàn khi chạy nhiều lần. Không cần migration tool cho Scope 1.

Map Terraform resource

Resource	File
`aws_lambda_function.ingester`	`modules/lambda/main.tf`
`aws_lambda_layer_version.pg8000`	`modules/lambda/main.tf`
`aws_lambda_event_source_mapping.sqs`	`modules/lambda/main.tf`
`aws_cloudwatch_log_group.lambda`	`modules/lambda/main.tf`
`aws_iam_role.lambda`	`modules/security/main.tf`
`aws_iam_role_policy.lambda_custom`	`modules/security/main.tf`

Health check & Logging

- **Health:** CloudWatch alarm `crawler-demo-lambda-errors` — Errors sum > 5 trong 5 phút → SNS email.
- **Log group:** `/aws/lambda/crawler-demo-ingester`, retention 30 ngày.
- **Structured logs:** JSON format với fields `event`, `trace_id`, `source`, `message_id`, `inserted`, `skipped`, `error`.

4.4 RDS PostgreSQL

Lý do chọn (so sánh alternatives)

Tiêu chí	RDS PostgreSQL ✅	DynamoDB	Aurora Serverless v2
Schema	Relational, typed	Schemaless	Relational
UNIQUE constraint	Native	Conditional write	Native
SQL queries	Full SQL	Limited	Full SQL
Chi phí	db.t3.micro ~$13/tháng	Pay-per-request	Min ~$43/tháng
Connection model	Connection-based	HTTP API	Connection-based
Free Tier	Có (db.t3.micro)	Có	Không
Scope 1 fit	✅ Tốt nhất	Không phù hợp	Quá đắt

Quyết định: PostgreSQL là lựa chọn tự nhiên cho dữ liệu có schema cố định, cần UNIQUE constraint cho idempotency, và cần SQL cho dashboard queries. DynamoDB không hỗ trợ UNIQUE constraint native. Aurora Serverless v2 có minimum capacity cost cao hơn.

Nguyên lý hoạt động

- **Engine:** PostgreSQL 15, `family = "postgres15"`.
- **Storage:** gp3 20GB, autoscale đến 100GB (`max_allocated_storage = 100`).
- **Encryption:** `storage_encrypted = true`, KMS CMK `alias/crawler-demo-key`.
- **Parameter group:** Custom `crawler-demo-pg15-params` với `log_connections=1`, `log_disconnections=1`, `log_min_duration_statement=1000` (log query > 1s).
- **Backup:** `backup_retention_period = 1` ngày, window `18:00-19:00 UTC`.
- **Maintenance:** `sun:19:00-sun:20:00 UTC`, `auto_minor_version_upgrade = true`.
- **Monitoring:** Performance Insights enabled (KMS encrypted), Enhanced Monitoring 60s interval.
- **Logs exported:** `postgresql`, `upgrade` → CloudWatch Logs.

Connection budget (quan trọng)

`db.t3.micro` có RAM 1GB. PostgreSQL mặc định `max_connections = LEAST(DBInstanceClassMemory/9531392, 5000)`.

```
max_connections ≈ 1024 MB × 1024 × 1024 / 9531392 ≈ 112 connections
```

Với safety factor 0.7: **~78 connections khả dụng**.

Consumer	Connections	Ghi chú
Lambda (max 5 concurrent)	5 × 1 = 5	Mỗi Lambda giữ 1 conn (warm)
Worker crawler-web	1–5	FastAPI connection pool
RDS Enhanced Monitoring	1	Internal
Admin/psql	1–2	Dự phòng
Tổng	**~15**	Rất an toàn với 78 khả dụng

Map Terraform resource

Resource	File
`aws_db_instance.main`	`modules/storage/main.tf`
`aws_db_subnet_group.main`	`modules/storage/main.tf`
`aws_db_parameter_group.main`	`modules/storage/main.tf`
`aws_iam_role.rds_monitoring`	`modules/storage/main.tf`

4.5 S3 (raw + exports)

Hai bucket với mục đích khác nhau

Bucket raw (`crawler-demo-raw-{account_id}`):

- **Mục đích:** Claim Check offload — lưu payload > 200 KiB tạm thời

- **Lifecycle:** Expire sau 30 ngày (đã ingested vào RDS)
- **Encryption:** SSE-KMS với CMK, `bucket_key_enabled = true` (giảm KMS API calls)
- **Access:** Worker write (`s3:PutObject`), Lambda read (`s3:GetObject`)
- **Public access:** Blocked hoàn toàn

Bucket exports (`crawler-demo-exports-{account_id}`):

- **Mục đích:** CSV/JSON exports từ dashboard
- **Versioning:** Enabled
- **Lifecycle:** Transition sang STANDARD_IA sau 30 ngày; noncurrent versions expire sau 90 ngày
- **Encryption:** SSE-KMS với CMK
- **Access:** Worker read (`s3:ListBucket` , `s3:GetObject`)
- **Public access:** Blocked hoàn toàn

S3 Gateway VPC Endpoint: Traffic từ private subnets đến S3 đi qua VPC Endpoint (free) thay vì NAT Gateway (tốn phí $0.045/GB). Đặc biệt quan trọng cho Claim Check pattern khi payload lớn.

Map Terraform resource

Resource	File
`aws_s3_bucket.raw`	`modules/storage/main.tf`
`aws_s3_bucket.exports`	`modules/storage/main.tf`
`aws_s3_bucket_server_side_encryption_configuration.*`	`modules/storage/main.tf`
`aws_s3_bucket_public_access_block.*`	`modules/storage/main.tf`
`aws_s3_bucket_lifecycle_configuration.*`	`modules/storage/main.tf`
`aws_vpc_endpoint.s3`	`modules/networking/main.tf`

4.6 ALB + FastAPI Dashboard

Kiến trúc

```
Internet → ALB (public subnets, sg_web_alb)
         → HTTP:80 Listener
         → Target Group (crawler-demo-web-tg)
         → EC2 Worker instances :8080
         → FastAPI app (crawler-web container)
         → RDS PostgreSQL (read queries)
```

ALB được định nghĩa trực tiếp trong `environments/demo/main.tf` (không phải submodule) vì nó phụ thuộc vào nhiều module outputs và là thành phần đặc thù của environment.

Security Group `sg_web_alb` :

- Ingress: TCP 80 từ `0.0.0.0/0`
- Egress: All

Security Group rule bổ sung cho Worker:

```
resource "aws_security_group_rule" "worker_web_from_alb" {
  type                     = "ingress"
  from_port                = var.web_port  # 8080
  to_port                  = var.web_port
  source_security_group_id = aws_security_group.web_alb.id
  security_group_id        = module.security.sg_worker_id
}
```

Rule này được thêm vào `sg_worker` từ environment level — không phải trong module security — để tránh circular dependency.

Health check: `GET /health` → HTTP 200, interval 30s, timeout 5s, healthy threshold 2, unhealthy threshold 2.

ASG attachment:

```
resource "aws_autoscaling_attachment" "web_tg" {
  autoscaling_group_name = module.worker.asg_name
  lb_target_group_arn    = aws_lb_target_group.web.arn
}
```

Khi ASG launch instance mới, ALB tự động register instance vào target group.

4.7 KMS + Secrets Manager

KMS Customer Managed Key (CMK)

Key policy có 2 statements:

1. `RootAccountFullAccess` : Account root có full access (recovery)
2. `AllowCloudWatchLogs` : CloudWatch Logs service có thể encrypt/decrypt (cho log group encryption)

Key rotation: `enable_key_rotation = true` — AWS tự động rotate key material hàng năm.

Deletion window: 7 ngày — nếu xóa nhầm có 7 ngày để cancel.

Alias: `alias/crawler-demo-key` — dùng alias thay vì ARN trong các resource để dễ reference.

Các resource được mã hóa bằng CMK:

- RDS storage (`kms_key_id`)
- SQS messages (`kms_master_key_id`)
- S3 objects (`kms_master_key_id`)
- Secrets Manager secret (`kms_key_id`)
- Performance Insights (`performance_insights_kms_key_id`)

Secrets Manager

Secret: `crawler-demo/db-credentials`

Format:

```
{
  "username": "crawler",
  "password": "<db_password>",
  "dbname": "crawlerdb",
  "engine": "postgres",
  "port": 5432
}
```

Lưu ý quan trọng: Lambda hiện tại đọc DB password từ **environment variable** (`DB_PASSWORD`) được inject lúc `terraform apply` , không phải từ Secrets Manager runtime. Secrets Manager được dùng như một secure store để Ansible và các tool khác có thể đọc credentials mà không cần hardcode trong plaintext.

`lifecycle { ignore_changes = [secret_string] }` : Terraform không overwrite secret nếu đã được rotate ngoài Terraform.

`recovery_window_in_days = 0` : Xóa ngay lập tức (phù hợp demo, không phù hợp production thực).

4.8 VPC + Networking

Xem chi tiết tại [Mục 5 — Mạng & bảo mật](#).

---

5. Mạng & bảo mật

5.1 Kiến trúc 3-tier subnet

```
┌──────────────────────────────────────────────────────────────┐
│  VPC: 10.0.0.0/16                                            │
│                                                              │
│  TIER 1 — PUBLIC (Internet-facing)                           │
│  ┌──────────────────────┐  ┌──────────────────────┐         │
│  │ public-1 AZ-a        │  │ public-2 AZ-b        │         │
│  │ 10.0.1.0/24          │  │ 10.0.2.0/24          │         │
│  │ NAT Gateway + EIP    │  │ ALB nodes            │         │
│  └──────────┬───────────┘  └──────────┬───────────┘         │
│             │ Route: 0.0.0.0/0 → IGW │                       │
│                                                              │
│  TIER 2 — PRIVATE (Workload)                                 │
│  ┌──────────────────────┐  ┌──────────────────────┐         │
│  │ private-1 AZ-a       │  │ private-2 AZ-b       │         │
│  │ 10.0.11.0/24         │  │ 10.0.12.0/24         │         │
│  │ EC2 Worker ASG       │  │ Lambda ENIs          │         │
│  └──────────┬───────────┘  └──────────┬───────────┘         │
│             │ Route: 0.0.0.0/0 → NAT │                       │
│             │ Route: S3 → VPC Endpoint                       │
│                                                              │
│  TIER 3 — DATABASE (Isolated)                                │
│  ┌──────────────────────┐  ┌──────────────────────┐         │
│  │ db-1 AZ-a            │  │ db-2 AZ-b            │         │
│  │ 10.0.21.0/24         │  │ 10.0.22.0/24         │         │
│  │ RDS (primary)        │  │ RDS (standby*)       │         │
```

```
│  └──────────────────────┘  └──────────────────────┘              │
│             │ Route: KHÔNG có internet route                      │
│             │ Route: S3 → VPC Endpoint                            │
└──────────────────────────────────────────────────────────────────┘
* Standby chỉ có ở Scope 2+ (Multi-AZ RDS)
```

Route tables:

- `rt-public` : `0.0.0.0/0 → aws_internet_gateway.main`
- `rt-private` : `0.0.0.0/0 → aws_nat_gateway.main` + S3 prefix → VPC Endpoint
- `rt-db` : Không có default route (isolated)

5.2 Security Groups — Rules chi tiết

sg_worker (`crawler-demo-sg-worker`)

Direction	Protocol	Port	Source/Dest	Mục đích
Egress	All	All	`0.0.0.0/0`	Crawl internet qua NAT, gửi SQS/S3
Ingress	TCP	8080	`sg_web_alb`	ALB forward đến FastAPI dashboard

Ingress rule được thêm từ `environments/demo/main.tf` qua `aws_security_group_rule.worker_web_from_alb` .

sg_lambda (`crawler-demo-sg-lambda`)

Direction	Protocol	Port	Source/Dest	Mục đích
Egress	All	All	`0.0.0.0/0`	Kết nối RDS (db subnet), S3 (VPC Endpoint)

Không có ingress rule — Lambda không nhận inbound connections.

sg_rds (`crawler-demo-sg-rds`)

Direction	Protocol	Port	Source/Dest	Mục đích
Ingress	TCP	5432	`sg_lambda`	Lambda Ingester đọc/ghi DB
Ingress	TCP	5432	`sg_worker`	Worker FastAPI dashboard đọc DB
Egress	All	All	`0.0.0.0/0`	Outbound (thực tế không dùng)

sg_web_alb (`crawler-demo-sg-web-alb`)

Direction	Protocol	Port	Source/Dest	Mục đích
Ingress	TCP	80	`0.0.0.0/0`	HTTP từ internet
Egress	All	All	`0.0.0.0/0`	Forward đến worker

5.3 IAM Least-Privilege

Lambda Execution Role (`crawler-demo-lambda-exec-role`)

Managed policies:

- `AWSLambdaBasicExecutionRole` : CloudWatch Logs write
- `AWSLambdaVPCAccessExecutionRole` : ENI create/delete cho VPC attachment

Custom policy `crawler-demo-lambda-custom-policy` :

Sid	Actions	Resource
`SQSConsume`	`ReceiveMessage`, `DeleteMessage`, `GetQueueAttributes`, `ChangeMessageVisibility`	`arn:aws:sqs:*:*:crawler-demo-*`
`S3RawReadClaimCheck`	`s3:GetObject`	`arn:aws:s3:::crawler-demo-raw-*/*`
`SecretsManagerRead`	`GetSecretValue`, `DescribeSecret`	`arn:aws:secretsmanager:*:*:secret:crawler-demo/*`
`KMSDecrypt`	`kms:Decrypt`, `kms:GenerateDataKey`	CMK ARN

Không có: `s3:PutObject` , `sqs:SendMessage` , `ec2:*` , `rds:*` — Lambda chỉ consume, không produce.

Worker Instance Role (`crawler-demo-worker-role`)

Managed policies:

- `AmazonSSMManagedInstanceCore` : SSM Session Manager (không cần SSH bastion)
- `AmazonEC2ContainerRegistryReadOnly` : ECR pull image
- `CloudWatchAgentServerPolicy` : CW Agent metrics + logs

Custom policy `crawler-demo-worker-custom-policy` :

Sid	Actions	Resource
`SQSSend`	`sqs:SendMessage`, `sqs:GetQueueUrl`	`arn:aws:sqs:*:*:crawler-demo-*`
`S3RawWriteClaimCheck`	`s3:PutObject`, `s3:AbortMultipartUpload`	`arn:aws:s3:::crawler-demo-raw-*/*`
`S3ExportsReadDashboard`	`s3:ListBucket`	`arn:aws:s3:::crawler-demo-exports-*`
`S3ExportsGetObjects`	`s3:GetObject`, `s3:GetObjectVersion`	`arn:aws:s3:::crawler-demo-exports-*/*`
`KMSEncrypt`	`kms:Encrypt`, `kms:GenerateDataKey`, `kms:Decrypt`	CMK ARN

Không có: `sqs:ReceiveMessage` , `sqs:DeleteMessage` — Worker chỉ produce, không consume.

5.4 Encryption at Rest và In Transit

Layer	At Rest	In Transit
RDS	KMS CMK (SSE)	SSL/TLS (pg8000 `ssl_context`)
SQS	KMS CMK	TLS-only policy (`aws:SecureTransport`)

S3 raw	KMS CMK (SSE-KMS)	HTTPS (S3 API)
S3 exports	KMS CMK (SSE-KMS)	HTTPS (S3 API)
Secrets Manager	KMS CMK	HTTPS (AWS API)
EC2 EBS	AES-256 (encrypted=true)	N/A
ALB → Worker	HTTP (Scope 1, no TLS)	Scope 2+: HTTPS

Lưu ý SSL trong Lambda:

```
ssl_ctx = ssl.create_default_context()
ssl_ctx.check_hostname = False
ssl_ctx.verify_mode = ssl.CERT_NONE
```

`CERT_NONE` được dùng vì pg8000 kết nối bằng hostname string và RDS cert được ký bởi AWS Global CA. Trong môi trường production nghiêm ngặt hơn, nên bundle AWS RDS CA cert và dùng `CERT_REQUIRED`.

5.5 IMDSv2

```
metadata_options {
  http_endpoint               = "enabled"
  http_tokens                 = "required"   # IMDSv2 bắt buộc
  http_put_response_hop_limit = 2            # Cho phép container truy cập IMDS
}
```

`hop_limit = 2` cần thiết vì Docker container cần 2 hops để đến IMDS endpoint (container → host → IMDS). Nếu để `hop_limit = 1`, container không thể lấy credentials từ instance profile.

5.6 Single NAT Gateway — SPOF Analysis

Scope 1 chọn single NAT để tiết kiệm chi phí (~$32/tháng/NAT). Đây là **SPOF có chủ đích** cho egress traffic:

- Nếu NAT Gateway fail (hiếm, AWS managed): Worker không thể crawl internet, không thể gửi SQS (SQS endpoint là public). Lambda vẫn hoạt động (đọc SQS qua VPC, kết nối RDS qua private network).
- **Scope 2+ mitigation:** Thêm NAT Gateway ở AZ-b, cập nhật `rt-private` per-AZ.

---

6. Infrastructure as Code — Terraform

6.1 Cấu trúc module và thứ tự phụ thuộc

```
environments/demo/main.tf
│
├── module.networking          (không phụ thuộc)
│   └── VPC, subnets, NAT, S3 endpoint
│
├── module.security            (depends_on: networking)
│   └── KMS, Secrets Manager, SGs, IAM roles
```

```
│
├── module.queue               (depends_on: security)
│   └── SQS main + DLQ
│
├── module.storage             (depends_on: networking, security)
│   └── RDS, S3 raw, S3 exports
│
├── module.lambda              (depends_on: storage, queue, security)
│   └── Lambda function, layer, ESM
│
├── module.worker              (depends_on: networking, security, queue, storage)
│   └── ECR, Launch Template, ASG
│
├── aws_lb + aws_lb_target_group + aws_lb_listener  (inline, depends on worker)
│   └── ALB, Target Group, Listener, SG rule
│
└── module.observability       (depends_on: lambda, storage, queue, worker)
    └── SNS, CW Alarms, Dashboard
```

Lý do thứ tự này:

- `security` phải có trước `queue` và `storage` vì cần KMS key ARN và SG IDs
- `storage` phải có trước `lambda` vì cần RDS endpoint
- `queue` phải có trước `lambda` vì cần SQS ARN cho ESM
- `worker` phải có trước `observability` vì cần ASG name cho alarm

6.2 Backend S3

```
# environments/demo/backend.tf
terraform {
  backend "s3" {
    bucket       = "crawler-terraform-state-478111025341"
    key          = "demo/terraform.tfstate"
    region       = "ap-southeast-1"
    use_lockfile = true   # S3 native locking (Terraform 1.10+), không cần DynamoDB
    encrypt      = true
  }
}
```

`use_lockfile = true` : Terraform 1.10+ hỗ trợ S3 native state locking bằng conditional writes — không cần DynamoDB table riêng. File lock: `demo/terraform.tfstate.tflock` .

Bucket naming: `crawler-terraform-state-{account_id}` — account ID trong tên đảm bảo globally unique.

6.3 Biến quan trọng

Biến	Default	Mô tả	Validation
`aws_region`	`ap-southeast-1`	AWS Region	—

`aws_account_id`	—	12-digit account ID	Required
`db_password`	—	RDS master password	`length >= 8`
`db_instance_class`	`db.t3.micro`	RDS instance class	`[db.t3.micro, db.t4g.micro]`
`ec2_instance_type`	`t3.micro`	Worker instance type	`[t3.micro, t4g.micro]`
`crawler_interval_seconds`	`1800`	Crawl cycle interval	—
`lambda_reserved_concurrency`	`null`	Lambda reserved concurrency	nullable
`lambda_event_source_max_concurrency`	`5`	ESM max concurrency	—
`web_port`	`8080`	FastAPI dashboard port	—
`alert_email`	—	SNS notification email	Required

6.4 Outputs quan trọng

Output	Mô tả	Sensitive
`rds_endpoint`	RDS hostname (cho Lambda, Ansible)	✅
`sqs_queue_url`	Main queue URL (cho Worker)	—
`ecr_repository_url`	ECR URL (cho CI/CD push)	—
`worker_asg_name`	ASG name (cho CI/CD refresh)	—
`nat_gateway_ip`	NAT EIP (whitelist ở crawl targets)	—
`web_dashboard_url`	`http://{alb_dns}`	—
`cloudwatch_dashboard_url`	CW dashboard URL	—
`db_secret_arn`	Secrets Manager ARN	—
`kms_key_arn`	CMK ARN	—

6.5 Lệnh chuẩn

```
# Khởi tạo (lần đầu hoặc sau khi thêm module)
terraform -chdir=infrastructure/terraform/environments/demo init

# Kiểm tra format
terraform -chdir=infrastructure/terraform/environments/demo fmt -check -recursive
```

```
# Validate
terraform -chdir=infrastructure/terraform/environments/demo validate

# Plan (xem thay đổi)
terraform -chdir=infrastructure/terraform/environments/demo plan \
  -var="aws_account_id=478111025341" \
  -var="db_password=$DB_PASSWORD" \
  -var="alert_email=ops@example.com"

# Apply
terraform -chdir=infrastructure/terraform/environments/demo apply \
  -var="aws_account_id=478111025341" \
  -var="db_password=$DB_PASSWORD" \
  -var="alert_email=ops@example.com"

# Xem outputs
terraform -chdir=infrastructure/terraform/environments/demo output

# Destroy (cẩn thận!)
terraform -chdir=infrastructure/terraform/environments/demo destroy
```

Truyền biến nhạy cảm qua environment:

```
export TF_VAR_db_password="$(aws secretsmanager get-secret-value \
  --secret-id crawler-demo/db-credentials \
  --query SecretString --output text | jq -r .password)"
```

6.6 Lưu ý quan trọng về Terraform

Thay đổi SG description: `aws_security_group` description là immutable — thay đổi sẽ force replace SG. Nếu RDS đang dùng SG đó, RDS sẽ mất kết nối trong thời gian replace. **Không thay đổi description của `sg_rds`.**

`apply_immediately = false` trên RDS: Thay đổi RDS (instance class, storage) sẽ được áp dụng trong maintenance window, không ngay lập tức. Tránh downtime ngoài kế hoạch.

`force_destroy = true` trên S3 raw: Bucket raw có thể bị xóa kể cả khi còn objects (phù hợp demo). Bucket exports có `force_destroy = false` — bảo vệ dữ liệu export.

`skip_final_snapshot = true` trên RDS: Không tạo snapshot khi destroy (phù hợp demo). Production nên đặt `false`.

---

7. Ansible — cấu hình runtime

7.1 Vai trò của Ansible vs Terraform

Khía cạnh	Terraform	Ansible
Mục đích	Provision infrastructure (IaC)	Configure runtime (Config Management)
State	Stateful (tfstate)	Stateless (idempotent tasks)

Khi chạy	Một lần khi tạo/thay đổi infra	Mỗi khi cần update config/image
Quản lý	AWS resources (EC2, RDS, SQS...)	OS packages, Docker, systemd units
Secrets	Terraform variables / env vars	Ansible Vault
Idempotency	Terraform plan/apply	Ansible module idempotency

Tại sao cần Ansible khi đã có `user_data`?

`user_data.sh.tpl` chỉ chạy **một lần** khi instance khởi động lần đầu. Ansible cho phép:

- Re-deploy config mà không cần terminate instance
- Update Docker image tag mà không cần instance refresh
- Thay đổi environment variables (ví dụ: crawl interval)
- Debug và fix config trên instance đang chạy

7.2 Cấu trúc Ansible

```
infrastructure/ansible/
├── ansible.cfg                    # default inventory, roles path
├── playbooks/
│   └── site.yml                   # main playbook
├── roles/
│   ├── base/                      # OS hardening, system packages
│   ├── docker/                    # Docker CE install trên AL2023
│   ├── cloudwatch_agent/          # CW Agent install + config
│   └── crawler_services/          # ECR login, docker pull, systemd units
└── inventory/
    ├── crawler_worker.aws_ec2.yml  # Dynamic inventory (AWS EC2 plugin)
    ├── demo-ssm.yml               # Static inventory qua SSM
    ├── demo.yml                   # Static inventory qua SSH
    └── group_vars/
        └── crawler_demo/
            ├── main.yml           # Non-secret vars
            ├── vault.yml          # Ansible Vault (DB password)
            └── connection.yml     # SSH/SSM connection settings
```

7.3 Các Ansible Role

Role: `base`

- Cập nhật OS packages (`dnf update`)
- OS hardening cơ bản
- Cài đặt các package hệ thống cần thiết

Role: `docker`

- Cài Docker CE trên Amazon Linux 2023
- Enable và start `docker.service`
- Thêm `ec2-user` vào group `docker`

Role: `cloudwatch_agent`

- Cài `amazon-cloudwatch-agent`

- Deploy config từ template `amazon-cloudwatch-agent.json.j2`
- Config log collection: `/var/log/crawler.log` → CW Log Group `/ec2/crawler-demo-worker`
- Start và enable CW Agent service

Role: `crawler_services`

Đây là role quan trọng nhất. Thực hiện theo thứ tự:

1. **Assert** biến bắt buộc (`crawler_web_db_password`)
2. **ECR login:** `aws ecr get-login-password | docker login` (`no_log: true`)
3. **Docker pull** với retry (3 lần, delay 10s)
4. **Tạo file log** `/var/log/crawler.log` (nếu chưa có)
5. **Deploy env file** `/etc/sysconfig/crawler-web` (mode 0600, `no_log: true`) — chứa DB password cho crawler-web container
6. **Deploy systemd units** từ Jinja2 templates:
   - `crawler-worker.service`
   - `crawler-web.service`
   - `crawler-log-forward.service`
7. **Copy script** `crawler-forward-docker-logs.sh`
8. **`systemctl daemon-reload`**
9. **Enable + restart** tất cả 3 services

7.4 Dynamic Inventory

```yaml
# inventory/crawler_worker.aws_ec2.yml
plugin: amazon.aws.aws_ec2
regions:
  - ap-southeast-1
filters:
  tag:Name: crawler-demo-worker
  instance-state-name: running
keyed_groups:
  - key: tags.Environment
    prefix: ""
```

Ansible tự động discover EC2 instances có tag `Name=crawler-demo-worker` đang running. Không cần hardcode IP.

Chạy với dynamic inventory:

```bash
cd infrastructure/ansible
ansible-playbook playbooks/site.yml --ask-vault-pass
```

Chạy với static inventory (SSH):

```bash
ansible-playbook -i inventory/demo.yml playbooks/site.yml --ask-vault-pass
```

Chạy với SSM (không cần SSH key):

```bash
ansible-playbook -i inventory/demo-ssm.yml playbooks/site.yml --ask-vault-pass
```

7.5 Ansible Vault

File: `inventory/group_vars/crawler_demo/vault.yml`

```
# Encrypted với ansible-vault
crawler_web_db_password: "<encrypted>"
```

Tạo vault:

```
ansible-vault create inventory/group_vars/crawler_demo/vault.yml
```

Edit vault:

```
ansible-vault edit inventory/group_vars/crawler_demo/vault.yml
```

Chạy playbook với vault:

```
ansible-playbook playbooks/site.yml --ask-vault-pass
# hoặc
ansible-playbook playbooks/site.yml --vault-password-file ~/.vault_pass
```

Lưu ý bảo mật: `vault.yml` được commit vào git ở dạng encrypted. `vault.yml.example` là template plaintext không chứa secret thực.

7.6 Systemd Services

`crawler-worker.service`

```
[Unit]
Description=Crawler Worker (Docker)
After=docker.service network-online.target
Requires=docker.service

[Service]
Restart=always
RestartSec=10
ExecStartPre=-/usr/bin/docker rm -f crawler-worker
ExecStart=/usr/bin/docker run --rm --name crawler-worker \
  -e CRAWLER_SCHEDULE_MODE=interval \
  -e CRAWLER_INTERVAL_SECONDS=1800 \
  -e CRAWLER_SQS_QUEUE_URL=https://sqs.ap-southeast-1.amazonaws.com/... \
  ...
  {ecr_repo}:{tag}
ExecStop=/usr/bin/docker stop crawler-worker
```

`crawler-web.service`

```
[Service]
ExecStart=/usr/bin/docker run --rm --name crawler-web \
  -p 8080:8080 \
  --env-file /etc/sysconfig/crawler-web \   # DB password từ file 0600
  {ecr_repo}:{tag} \
  uvicorn crawlerdemo.webapp:app --host 0.0.0.0 --port 8080 --app-dir src
```

Tại sao dùng `--env-file` thay vì `-e`? Tránh DB password xuất hiện trong `ps aux` output và systemd journal. File `/etc/sysconfig/crawler-web` có mode 0600 (chỉ root đọc được).

`crawler-log-forward.service`

Script `crawler-forward-docker-logs.sh` tail Docker logs của cả 2 container và append vào `/var/log/crawler.log`. CloudWatch Agent đọc file này và ship lên CW Logs.

7.7 So sánh user_data vs Ansible

Khía cạnh	user_data	Ansible
Khi chạy	Một lần (first boot)	Bất cứ lúc nào
Idempotency	Không (chạy lại = lỗi)	Có (module idempotent)
Debug	Xem `/var/log/user-data.log`	Verbose output trực tiếp
Update image	Cần instance refresh	`ansible-playbook`
Secrets	Terraform vars (plaintext trong state)	Ansible Vault
Scope 1 strategy	Bootstrap lần đầu	Re-configure sau đó

Thực tế trong project: `user_data.sh.tpl` bootstrap instance lần đầu (cài Docker, pull image, tạo systemd units). Ansible được dùng cho subsequent deployments và config changes mà không cần terminate instance.

---

8. CI/CD

8.1 Tổng quan pipeline

```
┌──────────────────────────────────────────────────────────────┐
│  GitHub Actions                                              │
│                                                              │
│  PR → main branch                                            │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  terraform-plan.yml                                  │    │
│  │  1. checkout                                         │    │
│  │  2. setup-terraform v3 (1.9.8)                       │    │
│  │  3. configure-aws-credentials (OIDC)                 │    │
│  │  4. terraform init + fmt-check + validate + plan     │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  Push → main branch                                          │
```

```
│  ┌──────────────────────────────────────────────────────────┐   │
│  │  deploy-aws.yml                                          │   │
│  │                                                          │   │
│  │  job: test ──────────────────────────────────────────┐  │   │
│  │    pytest -q                                         │  │   │
│  │                                                      │  │   │
│  │  job: terraform-validate ────────────────────────────┤  │   │
│  │    fmt-check + validate                              │  │   │
│  │                                                      ▼  │   │
│  │  job: build-and-push (needs: test, tf-validate) ─────┐  │   │
│  │    docker build --platform linux/amd64               │  │   │
│  │    docker push :sha + :latest                        │  │   │
│  │                                                      │  │   │
│  │  job: deploy-lambda (needs: tf-validate) ────────────┤  │   │
│  │    zip lambda_function.py                            │  │   │
│  │    aws lambda update-function-code --publish         │  │   │
│  │                                                      │  │   │
│  │  job: deploy-worker (needs: build-and-push) ─────────┘  │   │
│  │    wait for in-flight refresh                        │   │   │
│  │    start-instance-refresh (Rolling, 50% healthy)   │   │   │
│  │    poll until Successful/Failed/Cancelled            │   │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
```

8.2 OIDC Authentication

Không dùng long-lived AWS access keys. GitHub Actions dùng OIDC (OpenID Connect) để lấy temporary credentials:

```yaml
permissions:
  id-token: write   # Cần để request OIDC token
  contents: read

- name: Configure AWS credentials (OIDC)
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::478111025341:role/GitHubActionsRole
    aws-region: ap-southeast-1
```

IAM Role `GitHubActionsRole` cần trust policy cho GitHub OIDC provider:

```
{
  "Effect": "Allow",
  "Principal": {"Federated": "arn:aws:iam::478111025341:oidc-
provider/token.actions.githubusercontent.com"},
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {"token.actions.githubusercontent.com:aud":
"sts.amazonaws.com"},
    "StringLike": {"token.actions.githubusercontent.com:sub": "repo:org/repo:*"}
```

```
  }
}
```

8.3 Job: build-and-push

```
IMAGE="$ECR_REGISTRY/crawler-demo-worker:${{ github.sha }}"
docker build --platform linux/amd64 -t "$IMAGE" .
docker push "$IMAGE"
docker tag "$IMAGE" "$ECR_REGISTRY/crawler-demo-worker:latest"
docker push "$ECR_REGISTRY/crawler-demo-worker:latest"
```

Hai tags: `:sha` (immutable, cho rollback) và `:latest` (mutable, cho Ansible/user_data).

`--platform linux/amd64`: Đảm bảo image build đúng architecture kể cả khi CI runner là ARM (M1/M2 Mac).

8.4 Job: deploy-lambda

```
cd infrastructure/aws/lambda_ingester
zip -q lambda_function.zip lambda_function.py
aws lambda update-function-code \
  --function-name crawler-demo-ingester \
  --zip-file fileb://lambda_function.zip \
  --publish
```

Lambda deploy độc lập với Worker — không cần build Docker image. `--publish` tạo version mới (không ảnh hưởng đến ESM đang dùng `$LATEST` trừ khi có alias).

8.5 Job: deploy-worker (Rolling Instance Refresh)

```
# 1. Chờ nếu đang có refresh in-flight
wait_idle() {
  for i in $(seq 1 120); do
    STATUS=$(aws autoscaling describe-instance-refreshes ...)
    case "$STATUS" in
      Pending|InProgress) sleep 30 ;;
      *) return 0 ;;
    esac
  done
}

# 2. Start rolling refresh
aws autoscaling start-instance-refresh \
  --auto-scaling-group-name crawler-demo-worker-asg \
  --strategy Rolling \
  --preferences '{"MinHealthyPercentage":50,"InstanceWarmup":120}'

# 3. Poll đến khi Successful/Failed/Cancelled
```

Rolling strategy với `MinHealthyPercentage=50`: Với ASG desired=1, điều này có nghĩa là ASG sẽ launch instance mới trước khi terminate instance cũ (vì 0/1 = 0% < 50%). Thực tế: brief period có 2 instances.

`InstanceWarmup=120s`: Chờ 2 phút sau khi instance mới healthy trước khi tiếp tục refresh.

8.6 Rollback Strategy

Tình huống	Rollback
Lambda lỗi sau deploy	`aws lambda update-function-code --zip-file fileb://old.zip` hoặc dùng Lambda versions/aliases
Worker image lỗi	`docker pull {ecr}:{old_sha}` trên instance, hoặc update Launch Template image tag + instance refresh
Terraform apply lỗi	`terraform plan` để xem diff, `terraform apply` để fix, hoặc restore tfstate từ S3 versioning
RDS schema lỗi	Manual psql rollback (không có automated migration tool ở Scope 1)

Không có automated rollback trong Scope 1. Scope 2+ nên implement Lambda aliases với traffic shifting và ASG launch template version pinning.

8.7 Secrets trong CI/CD

Secret	Lưu ở	Dùng bởi
`AWS_ACCOUNT_ID`	GitHub Secrets	OIDC role ARN, ECR URL
`TF_VAR_DB_PASSWORD`	GitHub Secrets	terraform plan (PR)
DB password	Không lưu trong CI	Inject qua Ansible Vault khi chạy Ansible

---

9. Hoạch định năng lực & toán cấu hình

9.1 Visibility Timeout Math

Quy tắc: `VT ≥ 6 × Lambda_timeout`

```
Lambda timeout = 180s
VT = 6 × 180 = 1080s (18 phút)
```

Tại sao hệ số 6?

Nếu VT = Lambda_timeout, một Lambda đang xử lý gần hết timeout sẽ bị SQS re-deliver message trước khi Lambda kịp xóa. Hệ số 6 tạo buffer an toàn:

```
Timeline:
t=0:    Lambda nhận message, SQS ẩn message trong 1080s
t=180:  Lambda timeout (worst case) → Lambda crash
t=1080: SQS re-deliver message (ReceiveCount = 2)
Buffer: 1080 - 180 = 900s buffer
```

Nếu VT quá ngắn: Message bị re-deliver khi Lambda vẫn đang xử lý → duplicate processing → `ON CONFLICT DO NOTHING` xử lý được nhưng tốn tài nguyên.

Nếu VT quá dài: Message bị stuck lâu hơn khi Lambda crash thực sự → delay trước khi retry.

9.2 Connection Budget

db.t3.micro specs: 1 vCPU, 1 GB RAM

PostgreSQL max_connections formula:

```
max_connections = LEAST(DBInstanceClassMemory / 9531392, 5000)
               = LEAST(1073741824 / 9531392, 5000)
               = LEAST(112.6, 5000)
               ≈ 112 connections
```

Safety factor 0.7–0.8: Giữ lại buffer cho admin connections và spikes.

```
Usable connections = 112 × 0.75 = 84 connections
```

Connection allocation:

Consumer	Max connections	Ghi chú
Lambda (5 concurrent × 1 conn)	5	Warm start reuse, 1 conn/container
crawler-web FastAPI	5–10	Connection pool
RDS Enhanced Monitoring	1	Internal
Admin/psql sessions	2–3	Debug, migration
Tổng sử dụng	**~18**	
Buffer còn lại	**~66**	Rất thoải mái

Kết luận: db.t3.micro với `maximum_concurrency=5` là an toàn. Nếu tăng Lambda concurrency lên 20+, cần xem xét RDS Proxy.

9.3 Throughput Formula

Worker throughput:

```
Sources per cycle = N (số nguồn cấu hình)
Items per source  = max 100
Cycle interval    = 1800s

Max items/cycle   = N × 100
Max items/hour    = N × 100 × (3600/1800) = N × 200
```

SQS throughput:

```
Messages/cycle = N (1 message/source, có thể là inline hoặc claim check)
Message rate   = N / 1800 msg/s  (rất thấp)
```

Lambda throughput:

```
Batch size      = 10 records/invocation
Max concurrency = 5 invocations đồng thời
Max throughput  = 5 × 10 = 50 records/invocation-window

Lambda timeout  = 180s
Max records/min = 50 × (60/180) ≈ 16 records/min (conservative)
```

RDS write throughput:

```
INSERT rate = 50 records/invocation-window
Per-record  = ~1ms (simple INSERT với index)
Batch time  = 50ms cho 50 records (sequential trong 1 connection)
```

Bottleneck analysis: Ở Scope 1, bottleneck là Worker crawl rate (1800s interval), không phải Lambda hay RDS. Hệ thống có thể xử lý hàng nghìn articles/giờ nếu cần.

9.4 Claim Check Threshold Rationale

SQS message size limit: 256 KiB

Threshold: 200 KiB (`claim_check_threshold_bytes = 204800`)

Buffer: 256 - 200 = 56 KiB cho SQS metadata, message attributes, JSON overhead.

Tại sao không dùng 256 KiB? SQS tính size bao gồm message attributes và metadata. Để an toàn, threshold được đặt thấp hơn 20%.

Khi nào trigger Claim Check?

```
100 articles × ~2KB/article (title + summary + URL) = ~200KB
→ Gần ngưỡng → Claim Check được trigger thường xuyên với max_items=100
```

S3 traffic cost: Với S3 Gateway VPC Endpoint, không có NAT data processing fee. Chi phí chỉ là S3 PUT/GET requests (~$0.0004/1000 requests) và storage (expire sau 30 ngày).

9.5 ASG Scaling Math

Scale-out trigger: CPU > 70% trong 3 phút liên tiếp (3 × 60s periods)

Scale-in trigger: CPU < 40% trong 3 phút liên tiếp

Cooldown: 180s sau mỗi scaling action

Với desired=1, max=2:

```
Normal: 1 instance
Peak:   2 instances (sau scale-out)
Scale-in: 1 instance (sau 3 phút CPU < 40%)
```

Instance warmup: 120s (trong instance refresh) — thời gian để instance mới healthy trước khi nhận traffic.

9.6 ECR Lifecycle Policy

```
{
  "rules": [{
    "rulePriority": 1,
    "description": "Keep last 10 images",
    "selection": {"tagStatus": "any", "countType": "imageCountMoreThan",
"countNumber": 10},
    "action": {"type": "expire"}
  }]
}
```

Giữ 10 images gần nhất (theo push time). Với mỗi commit push 2 tags (`:sha` + `:latest`), thực tế giữ được ~5 commits gần nhất.

---

10. High Availability & Fault Tolerance

10.1 Trạng thái hiện tại (Scope 1)

Component	HA Status	SPOF?	Ghi chú
EC2 Worker ASG	Multi-AZ (AZ-a + AZ-b)	Không	ASG tự replace instance
SQS	AWS managed, Multi-AZ	Không	99.9% SLA
Lambda	AWS managed, Multi-AZ	Không	Serverless
RDS PostgreSQL	Single-AZ	**CÓ**	Scope 1 trade-off
NAT Gateway	Single (AZ-a)	**CÓ**	Scope 1 trade-off
S3	AWS managed, Multi-AZ	Không	99.999999999% durability
ALB	Multi-AZ	Không	AWS managed
KMS	AWS managed	Không	Regional service

10.2 SPOF Analysis

RDS Single-AZ

Rủi ro: Nếu AZ-a (nơi RDS đang chạy) có sự cố:

- Lambda không thể INSERT → messages quay lại SQS → retry → DLQ sau 3 lần
- Worker crawler-web không thể đọc DB → dashboard down
- Worker crawl vẫn hoạt động (chỉ gửi SQS, không đọc RDS)

RTO (Recovery Time Objective): ~5–10 phút (RDS restart trong AZ mới nếu AZ fail)

RPO (Recovery Point Objective): Tối đa 1 ngày (backup retention = 1 ngày)

Scope 2+ mitigation: `db_multi_az = true` trong `variables.tf` — một dòng thay đổi, ~$13/tháng thêm.

Single NAT Gateway

Rủi ro: Nếu NAT Gateway fail:

- Worker không thể crawl internet
- Worker không thể gửi SQS (SQS endpoint là public, đi qua NAT)
- Lambda vẫn hoạt động (đọc SQS qua VPC, kết nối RDS qua private network)
- Dashboard vẫn hoạt động (ALB → Worker → RDS, không qua NAT)

Scope 2+ mitigation: Thêm NAT Gateway ở AZ-b, tạo per-AZ route tables.

10.3 Cơ chế chống lỗi

Dead Letter Queue (DLQ)

```
Message → SQS Main Queue
  ↓ Lambda fail (exception, timeout)
  ↓ ReceiveCount++
  ↓ ReceiveCount = 3
  → SQS DLQ (crawler-demo-data-dlq)
  → CloudWatch Alarm (DLQ visible > 0)
  → SNS Email notification
```

Xử lý DLQ:

```
# Xem messages trong DLQ
aws sqs receive-message \
  --queue-url https://sqs.ap-southeast-1.amazonaws.com/478111025341/crawler-demo-
data-dlq \
  --max-number-of-messages 10

# Redrive về main queue (sau khi fix bug)
aws sqs start-message-move-task \
  --source-arn arn:aws:sqs:ap-southeast-1:478111025341:crawler-demo-data-dlq \
  --destination-arn arn:aws:sqs:ap-southeast-1:478111025341:crawler-demo-data-queue
```

Idempotency (Bất biến khi lặp lại)

Tầng SQS: At-least-once delivery — message có thể được deliver nhiều lần.

Tầng Lambda: `ReportBatchItemFailures` — chỉ failed records quay lại SQS.

Tầng DB: `INSERT ... ON CONFLICT (canonical_url) DO NOTHING` — duplicate URL không gây lỗi, không tạo duplicate row.

Kết quả: Hệ thống có thể xử lý cùng message nhiều lần mà không có side effects.

Partial Batch Failure

```
batch_item_failures = []
for record in records:
    try:
        # process record
        conn.commit()
    except Exception as exc:
        conn.rollback()  # clean state cho record tiếp theo
        batch_item_failures.append({"itemIdentifier": record["messageId"]})
```

```
return {"batchItemFailures": batch_item_failures}
```

Ví dụ: Batch 10 records, record #5 lỗi:

- Records 1–4: committed thành công, SQS xóa
- Record #5: rollback, thêm vào `batchItemFailures`, SQS giữ lại
- Records 6–10: tiếp tục xử lý, committed thành công, SQS xóa

Không có: "poison pill" làm fail cả batch.

Claim Check Pattern

Nếu Worker gửi payload lớn trực tiếp vào SQS và vượt 256 KiB → SQS reject. Claim Check pattern giải quyết:

1. Upload payload lên S3 (không giới hạn size)
2. Gửi pointer nhỏ vào SQS
3. Lambda fetch từ S3 khi xử lý

Fault tolerance: Nếu S3 object bị xóa trước khi Lambda xử lý → Lambda throw exception → message vào DLQ. S3 lifecycle 30 ngày đủ buffer.

ASG Auto-Recovery

```
Instance fail (EC2 status check fail)
  → ASG detect unhealthy
  → Terminate instance
  → Launch replacement instance (có thể ở AZ khác)
  → user_data bootstrap (Docker, systemd)
  → crawler-worker.service start
  → Tiếp tục crawl
```

Thời gian recovery: ~3–5 phút (EC2 launch + Docker pull + service start).

Trong thời gian recovery: SQS buffer messages, không mất dữ liệu.

10.4 Lộ trình Scope 2+

Cải tiến	Thay đổi	Chi phí thêm
RDS Multi-AZ	`db_multi_az = true`	~$13/tháng
NAT Gateway Multi-AZ	Thêm NAT ở AZ-b, per-AZ route tables	~$32/tháng
HTTPS trên ALB	ACM certificate + HTTPS listener	~$0 (ACM free)
RDS Proxy	Thêm `aws_db_proxy`	~$15/tháng
Lambda reserved concurrency	`lambda_reserved_concurrency = 10`	$0
Read replica	`aws_db_instance` với `replicate_source_db`	~$13/tháng
Multi-region	Phức tạp, cần thiết kế lại	Significant

---

11. Scale-up scenarios

11.1 Khi nào cần scale?

Dấu hiệu	Metric	Ngưỡng cảnh báo
Worker quá tải	EC2 CPU > 70% liên tục	ASG alarm đã có
Lambda throttle	Lambda Throttles > 0	Dashboard
DB connection exhaustion	RDS DatabaseConnections > 80	Thêm alarm
DB CPU cao	RDS CPUUtilization > 80%	Alarm đã có
SQS queue depth tăng	ApproximateNumberOfMessagesVisible > 100	Thêm alarm
DLQ tăng	DLQ visible > 0	Alarm đã có
ASG ở max	GroupInServiceInstances >= 2	Alarm đã có

11.2 Scale Worker ASG

Tăng max_size:

```
# environments/demo/main.tf
module "worker" {
  max_size         = 4  # tăng từ 2
  desired_capacity = 2  # tăng nếu cần
}
```

Tăng instance type:

```
variable "ec2_instance_type" {
  default = "t3.small"  # từ t3.micro
}
```

Lưu ý: Tăng instance type cần instance refresh (rolling). Tăng max_size không cần refresh.

11.3 Scale Lambda Concurrency

Tăng ESM max_concurrency:

```
variable "lambda_event_source_max_concurrency" {
  default = 10  # từ 5
}
```

Tăng memory (cũng tăng CPU):

```
lambda_memory_mb = 512  # từ 256
```

Lưu ý: Tăng concurrency → tăng DB connections. Kiểm tra connection budget trước.

11.4 Scale RDS

Upgrade instance class (vertical scale):

```
variable "db_instance_class" {
  default = "db.t3.small"  # từ db.t3.micro
}
```

→ `apply_immediately = false` → áp dụng trong maintenance window.

Thêm RDS Proxy (connection pooling):

```
resource "aws_db_proxy" "main" {
  name                   = "crawler-demo-proxy"
  engine_family          = "POSTGRESQL"
  role_arn               = aws_iam_role.rds_proxy.arn
  vpc_subnet_ids         = module.networking.private_subnet_ids
  vpc_security_group_ids = [module.security.sg_lambda_id]

  auth {
    auth_scheme = "SECRETS"
    secret_arn  = module.security.db_secret_arn
    iam_auth    = "DISABLED"
  }
}
```

RDS Proxy multiplexes connections — 100 Lambda connections → 5–10 actual DB connections.

Read replica (cho dashboard):

```
resource "aws_db_instance" "replica" {
  replicate_source_db = module.storage.rds_identifier
  instance_class      = "db.t3.micro"
  # ...
}
```

Trỏ `crawler-web` đến replica endpoint để giảm tải primary.

Multi-AZ:

```
# environments/demo/main.tf
module "storage" {
  db_multi_az = true  # từ false
}
```

11.5 Scale SQS Throughput

SQS Standard gần như không có giới hạn throughput thực tế. Không cần action ở Scope 1–3.

Nếu cần ordering hoặc exactly-once: migrate sang SQS FIFO (cần thay đổi code Worker và Lambda).

11.6 Multi-region Considerations

Scope 1–2 không cần multi-region. Nếu cần:

Component	Multi-region strategy
RDS	Cross-region read replica hoặc Aurora Global Database
S3	Cross-region replication
SQS	Không có native replication — cần application-level routing
Lambda	Deploy riêng mỗi region
ECR	Replicate image sang region khác
Route 53	Latency-based routing hoặc failover

---

12. Lỗi thường gặp & troubleshooting

12.1 Bảng triệu chứng → nguyên nhân → cách tra → fix

#	Triệu chứng	Nguyên nhân có thể	Cách tra	Fix
1	Lambda timeout	DB slow query, connection pool exhausted, payload quá lớn	CW Logs `/aws/lambda/crawler-demo-ingester` tìm `record_failed` + `error`	Tăng timeout, optimize query, kiểm tra DB connections
2	DLQ tăng	Lambda lỗi liên tục, DB down, parse error	CW Alarm email + xem DLQ messages	Fix bug, redrive sau khi fix
3	Too many DB connections	Lambda concurrency quá cao, connection leak	RDS `DatabaseConnections` metric, `SELECT count(*) FROM pg_stat_activity`	Giảm `maximum_concurrency`, thêm RDS Proxy
4	Worker không pull image	ECR auth expired, IAM permission thiếu, NAT down	SSH/SSM vào instance, `docker pull` thủ công, xem `/var/log/user-data.log`	`aws ecr get-login-password
5	NAT cost spike	Claim Check traffic qua NAT thay vì VPC Endpoint	VPC Flow Logs, Cost Explorer	Kiểm tra S3 VPC Endpoint route table associations

6	Terraform state lock	CI/CD bị interrupt, lock file còn lại	`terraform force-unlock <lock_id>`	Xóa lock file trong S3: `demo/terraform.tfstate.tflock`
7	SQS VT mismatch	Lambda timeout > VT/6	So sánh `visibility_timeout_seconds` vs `lambda_timeout_seconds × 6`	Tăng VT hoặc giảm Lambda timeout
8	Lambda cold start chậm	VPC ENI provisioning	CW Logs `INIT_START` duration	Tăng `minimum_concurrency` (provisioned concurrency)
9	RDS CPU cao	N+1 queries từ dashboard, missing index	Performance Insights, `pg_stat_statements`	Thêm index, optimize query, thêm read replica
10	ASG không scale out	CPU alarm không trigger, cooldown active	CW Alarm history, ASG activity log	Kiểm tra alarm dimensions, chờ cooldown
11	Dashboard 502/503	crawler-web container crash, ALB health check fail	ALB target group health, `docker ps` trên instance	Restart service, kiểm tra DB connection
12	Duplicate articles	`canonical_url` không chuẩn hóa	`SELECT canonical_url, count(*) FROM articles GROUP BY 1 HAVING count(*) > 1`	Fix normalization trong Worker code

12.2 Lambda Timeout (chi tiết)

Triệu chứng: CW Alarm `crawler-demo-lambda-errors` trigger, log có `Task timed out after 180.00 seconds`.

Điều tra:

```
# Xem logs gần nhất
aws logs filter-log-events \
  --log-group-name /aws/lambda/crawler-demo-ingester \
  --filter-pattern "record_failed" \
  --start-time $(date -d '1 hour ago' +%s000)

# Xem batch summary
aws logs filter-log-events \
  --log-group-name /aws/lambda/crawler-demo-ingester \
  --filter-pattern "batch_summary"
```

Nguyên nhân phổ biến:

1. DB connection timeout (RDS restart, network issue)
2. S3 GetObject chậm (Claim Check với object lớn)
3. Batch quá nhiều articles (100 articles × slow INSERT)

Fix:

- Kiểm tra RDS status: `aws rds describe-db-instances --db-instance-identifier crawler-demo-db`
- Giảm `batch_size` từ 10 xuống 5
- Tăng Lambda timeout (max 900s cho SQS trigger)

12.3 Too Many DB Connections (chi tiết)

Triệu chứng: Lambda log `FATAL: remaining connection slots are reserved for non-replication superuser connections`.

Điều tra:

```
-- Kết nối qua SSM → psql
SELECT client_addr, state, count(*)
FROM pg_stat_activity
GROUP BY client_addr, state
ORDER BY count(*) DESC;

SELECT count(*) FROM pg_stat_activity;
SELECT setting FROM pg_settings WHERE name = 'max_connections';
```

Fix ngắn hạn:

```
# Giảm Lambda concurrency ngay lập tức
aws lambda put-function-concurrency \
  --function-name crawler-demo-ingester \
  --reserved-concurrent-executions 2
```

Fix dài hạn: Thêm RDS Proxy (xem mục 11.4).

12.4 DLQ Tăng (chi tiết)

Triệu chứng: Email từ SNS `crawler-demo-alerts`, subject `ALARM: crawler-demo-dlq-has-messages`.

Điều tra:

```
# Xem message trong DLQ
aws sqs receive-message \
  --queue-url https://sqs.ap-southeast-1.amazonaws.com/478111025341/crawler-demo-data-dlq \
  --max-number-of-messages 1 \
  --attribute-names All

# Xem Lambda errors
```

```
aws logs filter-log-events \
  --log-group-name /aws/lambda/crawler-demo-ingester \
  --filter-pattern "record_failed" \
  --start-time $(date -d '2 hours ago' +%s000)
```

Redrive sau khi fix:

```
aws sqs start-message-move-task \
  --source-arn arn:aws:sqs:ap-southeast-1:478111025341:crawler-demo-data-dlq \
  --destination-arn arn:aws:sqs:ap-southeast-1:478111025341:crawler-demo-data-queue
\
  --max-number-of-messages-per-second 1
```

12.5 Terraform State Issues (chi tiết)

Lock stuck:

```
# Xem lock file
aws s3 ls s3://crawler-terraform-state-478111025341/demo/

# Force unlock (cẩn thận!)
terraform -chdir=infrastructure/terraform/environments/demo \
  force-unlock <lock_id>

# Hoặc xóa lock file trực tiếp
aws s3 rm s3://crawler-terraform-state-478111025341/demo/terraform.tfstate.tflock
```

State drift (resource bị xóa ngoài Terraform):

```
# Import lại resource
terraform -chdir=infrastructure/terraform/environments/demo \
  import module.storage.aws_db_instance.main crawler-demo-db

# Hoặc refresh state
terraform -chdir=infrastructure/terraform/environments/demo refresh
```

12.6 Worker Không Pull Image (chi tiết)

Điều tra qua SSM:

```
# Kết nối SSM
aws ssm start-session --target <instance-id>

# Kiểm tra Docker
sudo docker ps -a
sudo systemctl status crawler-worker.service
sudo journalctl -u crawler-worker.service -n 50

# Test ECR auth thủ công
aws ecr get-login-password --region ap-southeast-1 \
```

```
  | sudo docker login --username AWS --password-stdin \
    478111025341.dkr.ecr.ap-southeast-1.amazonaws.com

# Test pull
sudo docker pull 478111025341.dkr.ecr.ap-southeast-1.amazonaws.com/crawler-demo-
worker:latest
```

Nguyên nhân phổ biến:

1. ECR auth token expired (valid 12h) → `aws ecr get-login-password` lại
2. IAM role thiếu `ecr:GetAuthorizationToken` → kiểm tra `AmazonEC2ContainerRegistryReadOnly`
3. NAT Gateway down → không thể reach ECR endpoint
4. Image chưa được push → kiểm tra CI/CD job `build-and-push`

---

13. Observability chi tiết

13.1 Log Groups và Format

Lambda: `/aws/lambda/crawler-demo-ingester`

Structured JSON logging:

```
// record_processed
{
  "event": "record_processed",
  "trace_id": "abc-123",
  "source": "vnexpress.net",
  "message_id": "msg-uuid-xxx",
  "inserted": 8,
  "skipped": 2
}

// record_failed
{
  "event": "record_failed",
  "trace_id": "abc-123",
  "message_id": "msg-uuid-yyy",
  "error": "connection refused"
}

// batch_summary
{
  "event": "batch_summary",
  "records": 10,
  "inserted": 45,
  "skipped": 5,
  "failed": 0
}
```

Log levels: INFO (default), ERROR (failures). Có thể thay đổi qua `LOG_LEVEL` env var.

Retention: 30 ngày.

Worker: `/ec2/crawler-demo-worker`

Log streams:

- `{instance_id}/user-data` : Bootstrap logs từ `user_data.sh`
- `{instance_id}/crawler` : Application logs từ Docker containers (qua `crawler-log-forward.service`)

Format: Phụ thuộc vào application code trong Docker image.

Retention: 30 ngày.

RDS Logs

- `postgresql` : Connection logs, slow queries (> 1s), errors
- `upgrade` : Engine upgrade events

Exported tự động sang CloudWatch Logs qua `enabled_cloudwatch_logs_exports` .

13.2 CloudWatch Alarms

Alarm	Metric	Namespace	Threshold	Period	Eval
`crawler-demo-dlq-has-messages`	`ApproximateNumberOfMessagesVisible`	`AWS/SQS`	> 0	60s	1
`crawler-demo-lambda-errors`	`Errors` (Sum)	`AWS/Lambda`	> 5	300s	1
`crawler-demo-rds-cpu-high`	`CPUUtilization` (Avg)	`AWS/RDS`	> 80%	60s	5
`crawler-demo-worker-asg-at-max`	`GroupInServiceInstances` (Max)	`AWS/AutoScaling`	>= 2	300s	2
`crawler-demo-worker-cpu-high`	`CPUUtilization` (Avg)	`AWS/EC2`	> 70%	60s	3
`crawler-demo-worker-cpu-low`	`CPUUtilization` (Avg)	`AWS/EC2`	< 40%	60s	3

Lý do các ngưỡng:

- **DLQ > 0:** Bất kỳ message nào vào DLQ đều cần điều tra ngay. Không có false positive.
- **Lambda Errors > 5 / 5min:** Cho phép 1–2 transient errors mà không alarm. > 5 là dấu hiệu systemic issue.
- **RDS CPU > 80% / 5min:** 80% là ngưỡng trước khi performance degradation nghiêm trọng. 5 phút tránh false positive từ spike ngắn.
- **ASG at max / 10min:** Cảnh báo cần tăng `max_size` hoặc optimize Worker.

13.3 CloudWatch Dashboard: `crawler-demo-overview`

Layout (4 widgets, 2×2 grid):

```
┌──────────────────────────┬──────────────────────────┐
│  SQS — Messages          │  Lambda — Invoc/Err/Thr  │
│  Main queue (blue)       │  Invocations (blue)      │
│  DLQ (red)               │  Errors (red)            │
│  Period: 60s, Max        │  Throttles (orange)      │
│                          │  Period: 60s, Sum        │
├──────────────────────────┼──────────────────────────┤
│  RDS — CPU & Conns       │  Worker ASG — CPU & Cap  │
│  CPUUtilization (blue)   │  EC2 CPU % (blue)        │
│  DatabaseConnections     │  GroupInServiceInstances │
│  Period: 60s, Avg        │  (right axis, orange)    │
│                          │  Period: 60s, Avg        │
└──────────────────────────┴──────────────────────────┘
```

URL: `https://ap-southeast-1.console.aws.amazon.com/cloudwatch/home#dashboards:name=crawler-demo-overview`

13.4 Performance Insights

RDS Performance Insights enabled với KMS encryption. Truy cập:

```
AWS Console → RDS → crawler-demo-db → Performance Insights
```

Metrics hữu ích:

- `db.load.avg` : Average active sessions
- `db.sql.calls` : Top SQL statements by call count
- `db.sql.avg_latency` : Slow queries
- `db.wait_event` : Wait events (lock, I/O, CPU)

13.5 Correlation Tracing

Hệ thống hiện tại không có distributed tracing (X-Ray disabled — `tracing_config.mode = "PassThrough"`). Correlation được thực hiện qua:

1. **`trace_id`** trong SQS message attributes → Lambda log `record_processed` / `record_failed`
2. **`message_id`** (SQS message ID) → Lambda log
3. **`source`** field → filter logs theo nguồn

Ví dụ correlation query (CloudWatch Logs Insights):

```
# Tìm tất cả records từ một nguồn trong 1 giờ qua
fields @timestamp, event, source, inserted, skipped, error
| filter source = "vnexpress.net"
| sort @timestamp desc
| limit 100
```

```
# Tìm tất cả failures
fields @timestamp, event, message_id, error
| filter event = "record_failed"
| sort @timestamp desc
| limit 50
```

```
# Tổng hợp throughput theo giờ
stats sum(inserted) as total_inserted, sum(skipped) as total_skipped
  by bin(1h)
| filter event = "batch_summary"
```

13.6 Enhanced Monitoring

RDS Enhanced Monitoring (60s interval) cung cấp OS-level metrics:

- `cpuUtilization.total`
- `memory.free`
- `diskIO.readIOsPS` , `diskIO.writeIOsPS`
- `network.rx` , `network.tx`

Metrics xuất hiện trong CloudWatch namespace `CWAgent` với dimension `InstanceId` .

---

14. Checklist vận hành

14.1 Trước khi Deploy (Pre-deploy)

Infrastructure (Terraform)

- [ ] `terraform fmt -check -recursive` — không có format error
- [ ] `terraform validate` — không có syntax error
- [ ] `terraform plan` — review thay đổi, không có unexpected destroy
- [ ] Kiểm tra `db_multi_az` , `deletion_protection` , `skip_final_snapshot` phù hợp với môi trường
- [ ] Xác nhận `alert_email` đúng và đã confirm SNS subscription
- [ ] Kiểm tra S3 backend bucket tồn tại và có versioning enabled
- [ ] Không có sensitive values trong plan output (dùng `sensitive = true`)

Application (Docker Image)

- [ ] Unit tests pass (`pytest -q`)
- [ ] Docker build thành công (`docker build --platform linux/amd64`)
- [ ] Image scan ECR không có critical vulnerabilities
- [ ] Lambda function zip đúng file (`lambda_function.py`)

Ansible

- [ ] `ansible-vault` secrets đã được update nếu có thay đổi
- [ ] Dynamic inventory hoạt động: `ansible-inventory --list`
- [ ] Dry-run: `ansible-playbook --check playbooks/site.yml`

14.2 Trong khi Deploy

Terraform Apply

- [ ] Monitor `terraform apply` output — không có unexpected errors
- [ ] Nếu RDS thay đổi: xác nhận `apply_immediately = false` (áp dụng trong maintenance window)
- [ ] Sau apply: `terraform output` để lấy endpoints mới

CI/CD Deploy

- [ ] Monitor GitHub Actions jobs: `test` → `terraform-validate` → `build-and-push` → `deploy-lambda` → `deploy-worker`
- [ ] Kiểm tra ECR: image mới có tag `:sha` và `:latest`
- [ ] Kiểm tra Lambda: function code updated, version mới published
- [ ] Monitor ASG instance refresh: `aws autoscaling describe-instance-refreshes --auto-scaling-group-name crawler-demo-worker-asg`
- [ ] Kiểm tra ALB target group health: tất cả instances healthy

Ansible Deploy

- [ ] Monitor playbook output — không có `failed=` > 0
- [ ] Kiểm tra services: `systemctl status crawler-worker crawler-web crawler-log-forward`
- [ ] Kiểm tra Docker containers: `docker ps`

14.3 Sau khi Deploy (Post-deploy)

Smoke Tests

- [ ] Dashboard accessible: `curl http://{alb_dns}/health` → 200
- [ ] Lambda test invoke:

```
aws lambda invoke \
  --function-name crawler-demo-ingester \
  --payload '{"action":"init-schema"}' \
  response.json
cat response.json  # {"schema_ready": true}
```

- [ ] SQS: gửi test message và kiểm tra Lambda xử lý
- [ ] RDS: kiểm tra articles table có data mới

Monitoring

- [ ] CloudWatch Dashboard `crawler-demo-overview` — không có anomaly
- [ ] DLQ empty: `aws sqs get-queue-attributes --queue-url {dlq_url} --attribute-names ApproximateNumberOfMessagesVisible`
- [ ] Lambda errors = 0 trong 15 phút đầu
- [ ] RDS CPU bình thường (< 50%)

- [ ] Không có SNS alarm email

Rollback readiness

- [ ] Ghi lại image SHA trước deploy (để rollback nếu cần)
- [ ] Ghi lại Lambda version trước deploy

14.4 Vận hành định kỳ

Hàng ngày

- [ ] Kiểm tra CloudWatch Dashboard
- [ ] Kiểm tra DLQ empty
- [ ] Kiểm tra Lambda error rate

Hàng tuần

- [ ] Review CloudWatch Logs Insights: throughput, error patterns
- [ ] Kiểm tra RDS storage: `aws rds describe-db-instances --query 'DBInstances[0].AllocatedStorage'`
- [ ] Kiểm tra ECR lifecycle: không quá 10 images
- [ ] Review Cost Explorer: NAT Gateway data transfer, Lambda invocations, RDS

Hàng tháng

- [ ] Review và rotate DB password (update Secrets Manager + Ansible Vault + Terraform var)
- [ ] Kiểm tra KMS key rotation status
- [ ] Review IAM policies — không có over-privileged permissions
- [ ] Test RDS backup restore (nếu production)
- [ ] Review S3 raw bucket: lifecycle đang hoạt động, không có stale objects
- [ ] Update dependencies: Terraform provider version, Lambda runtime, Docker base image

Khi có incident

- [ ] Kiểm tra DLQ messages — xem error details
- [ ] Kiểm tra Lambda logs — filter `record_failed`
- [ ] Kiểm tra RDS Performance Insights
- [ ] Kiểm tra ASG activity log
- [ ] Nếu cần rollback: xem mục 8.6

14.5 Lệnh vận hành thường dùng

```
# Xem trạng thái ASG
aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names crawler-demo-worker-asg \
  --query 'AutoScalingGroups[0].
{Desired:DesiredCapacity,Min:MinSize,Max:MaxSize,Instances:Instances[*].
{ID:InstanceId,State:LifecycleState,Health:HealthStatus}}'

# Xem Lambda metrics gần nhất
aws cloudwatch get-metric-statistics \
  --namespace AWS/Lambda \
  --metric-name Errors \
```

```
  --dimensions Name=FunctionName,Value=crawler-demo-ingester \
  --start-time $(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 300 \
  --statistics Sum

# Xem SQS queue depth
aws sqs get-queue-attributes \
  --queue-url https://sqs.ap-southeast-1.amazonaws.com/478111025341/crawler-demo-
data-queue \
  --attribute-names
ApproximateNumberOfMessagesVisible,ApproximateNumberOfMessagesNotVisible

# Kết nối SSM vào worker instance
INSTANCE_ID=$(aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=crawler-demo-worker" "Name=instance-state-
name,Values=running" \
  --query 'Reservations[0].Instances[0].InstanceId' --output text)
aws ssm start-session --target $INSTANCE_ID

# Xem articles count
# (qua SSM → psql)
psql "host=$(terraform -chdir=infrastructure/terraform/environments/demo output -raw
rds_endpoint) \
  user=crawler dbname=crawlerdb" \
  -c "SELECT source, count(*) FROM articles GROUP BY source ORDER BY count(*) DESC;"

# Force Lambda cold start (update env var)
aws lambda update-function-configuration \
  --function-name crawler-demo-ingester \
  --environment "Variables=
{LOG_LEVEL=DEBUG,RDS_HOST=...,DB_NAME=crawlerdb,DB_USER=crawler,DB_PASSWORD=...}"
```

---

15. Phụ lục — Quick Reference Map

15.1 Khái niệm SA → Terraform Resource → Ansible Role

Khái niệm SA	Terraform Resource	Module/File	Ansible Role
VPC	`aws_vpc.main`	`modules/networking`	—
Public Subnet	`aws_subnet.public[*]`	`modules/networking`	—
Private Subnet	`aws_subnet.private[*]`	`modules/networking`	—
DB Subnet	`aws_subnet.db[*]`	`modules/networking`	—

NAT Gateway	`aws_nat_gateway.main`	`modules/networking`	—
Internet Gateway	`aws_internet_gateway.main`	`modules/networking`	—
S3 VPC Endpoint	`aws_vpc_endpoint.s3`	`modules/networking`	—
KMS CMK	`aws_kms_key.main`	`modules/security`	—
Secrets Manager	`aws_secretsmanager_secret.db`	`modules/security`	—
SG Worker	`aws_security_group.worker`	`modules/security`	—
SG Lambda	`aws_security_group.lambda`	`modules/security`	—
SG RDS	`aws_security_group.rds`	`modules/security`	—
SG ALB	`aws_security_group.web_alb`	`environments/demo/main.tf`	—
Lambda IAM Role	`aws_iam_role.lambda`	`modules/security`	—
Worker IAM Role	`aws_iam_role.worker`	`modules/security`	—
Worker IAM Profile	`aws_iam_instance_profile.worker`	`modules/security`	—
SQS Main Queue	`aws_sqs_queue.main`	`modules/queue`	—
SQS DLQ	`aws_sqs_queue.dlq`	`modules/queue`	—
SQS TLS Policy	`aws_sqs_queue_policy.main/dlq`	`modules/queue`	—
RDS PostgreSQL	`aws_db_instance.main`	`modules/storage`	—
RDS Subnet Group	`aws_db_subnet_group.main`	`modules/storage`	—
RDS Parameter Group	`aws_db_parameter_group.main`	`modules/storage`	—

S3 Raw Bucket	`aws_s3_bucket.raw`	`modules/storage`	—
S3 Exports Bucket	`aws_s3_bucket.exports`	`modules/storage`	—
Lambda Function	`aws_lambda_function.ingester`	`modules/lambda`	—
Lambda Layer	`aws_lambda_layer_version.pg8000`	`modules/lambda`	—
Lambda ESM	`aws_lambda_event_source_mapping.sqs`	`modules/lambda`	—
Lambda Log Group	`aws_cloudwatch_log_group.lambda`	`modules/lambda`	—
ECR Repository	`aws_ecr_repository.worker`	`modules/worker`	—
Launch Template	`aws_launch_template.worker`	`modules/worker`	—
ASG	`aws_autoscaling_group.worker`	`modules/worker`	—
ASG Scale-out Policy	`aws_autoscaling_policy.scale_out`	`modules/worker`	—
ASG Scale-in Policy	`aws_autoscaling_policy.scale_in`	`modules/worker`	—
Worker Log Group	`aws_cloudwatch_log_group.worker`	`modules/worker`	—
SNS Alert Topic	`aws_sns_topic.alerts`	`modules/observability`	—
CW Dashboard	`aws_cloudwatch_dashboard.main`	`modules/observability`	—
ALB	`aws_lb.web`	`environments/demo/main.tf`	—
ALB Target Group	`aws_lb_target_group.web`	`environments/demo/main.tf`	—
ALB Listener	`aws_lb_listener.web_http`	`environments/demo/main.tf`	—
OS hardening	—	—	`base`
Docker CE	—	—	`docker`

CW Agent	—	—	`cloudwatch_agent`
crawler-worker service	—	—	`crawler_services`
crawler-web service	—	—	`crawler_services`
crawler-log-forward service	—	—	`crawler_services`
ECR login	—	—	`crawler_services`
Docker pull	—	—	`crawler_services`

15.2 Environment Variables Map

Env Var	Container	Nguồn giá trị	Terraform Output
`CRAWLER_SQS_QUEUE_URL`	crawler-worker	`module.queue.main_queue_url`	`sqs_queue_url`
`CRAWLER_S3_RAW_BUCKET`	crawler-worker	`module.storage.s3_raw_bucket`	`s3_raw_bucket`
`CRAWLER_INTERVAL_SECONDS`	crawler-worker	`var.crawler_interval_seconds` (1800)	—
`CRAWLER_MAX_ITEMS_PER_SOURCE`	crawler-worker	hardcoded 100	—
`CRAWLER_CLAIM_CHECK_THRESHOLD_BYTES`	crawler-worker	hardcoded 204800	—
`CRAWLER_AWS_REGION`	crawler-worker	`var.aws_region`	—
`WEB_DB_HOST`	crawler-web	`module.storage.rds_endpoint`	`rds_endpoint`
`WEB_DB_PORT`	crawler-web	`module.storage.rds_port` (5432)	`rds_port`
`WEB_DB_NAME`	crawler-web	`module.storage.db_name` (crawlerdb)	`db_name`
`WEB_DB_USER`	crawler-web	`module.storage.db_username` (crawler)	`db_username`

`WEB_DB_PASSWORD`	crawler-web	`var.db_password` (sensitive)	—
`WEB_S3_EXPORTS_BUCKET`	crawler-web	`module.storage.s3_exports_bucket`	`s3_exports_bu`
`RDS_HOST`	Lambda	`module.storage.rds_endpoint`	`rds_endpoint`
`DB_NAME`	Lambda	`module.storage.db_name`	`db_name`
`DB_USER`	Lambda	`module.storage.db_username`	`db_username`
`DB_PASSWORD`	Lambda	`var.db_password` (sensitive)	—
`LOG_LEVEL`	Lambda	hardcoded `INFO`	—

15.3 Port Map

Port	Protocol	From	To	Mục đích
80	HTTP	Internet	ALB	Dashboard access
8080	HTTP	ALB	Worker EC2	FastAPI dashboard
5432	TCP	Lambda ENI	RDS	Lambda → PostgreSQL
5432	TCP	Worker EC2	RDS	crawler-web → PostgreSQL
443	HTTPS	Worker EC2	SQS/S3/ECR/SSM	AWS API calls (qua NAT)

15.4 Naming Convention

Resource type	Pattern	Ví dụ
Tất cả resources	`{project}-{environment}-{resource}`	`crawler-demo-vpc`
SQS queues	`crawler-demo-data-queue`, `crawler-demo-data-dlq`	
S3 buckets	`crawler-demo-{purpose}-{account_id}`	`crawler-demo-raw-478111025341`
IAM roles	`crawler-demo-{service}-{type}-role`	`crawler-demo-lambda-exec-role`
Security groups	`crawler-demo-sg-{service}`	`crawler-demo-sg-worker`
Log groups	`/aws/lambda/{function}`, `/ec2/{asg}`	`/aws/lambda/crawler-demo-ingester`
KMS alias	`alias/{project}-{environment}-key`	`alias/crawler-demo-key`
Secrets	`{project}-{environment}/{secret}`	`crawler-demo/db-credentials`

Tags	`Project=crawler`, `Environment=demo`, `ManagedBy=Terraform`, `Scope=scope-1`	

15.5 Tài liệu tham khảo

Tài liệu	Đường dẫn
Architecture overview	`docs/ARCHITECTURE.md`
Terraform-Service map	`docs/SERVICES_AND_TERRAFORM.md`
SA-Terraform map	`docs/ARCHITECTURE_SOLUTION_TERRAFORM_MAP.md`
Terraform README	`infrastructure/terraform/README.md`
Ansible README	`infrastructure/ansible/README.md`
Lambda function	`infrastructure/aws/lambda_ingester/lambda_function.py`
DB Schema	`infrastructure/aws/lambda_ingester/schema.sql`
Demo environment	`infrastructure/terraform/environments/demo/main.tf`

Tài liệu này là nguồn sự thật duy nhất (single source of truth) cho kiến trúc hệ thống Web Crawler & Ingestion Pipeline. Mọi thay đổi kiến trúc phải được phản ánh trong tài liệu này.

Cập nhật lần cuối: Phiên bản 1.0 — Scope 1 Production-Ready